DOCUMENTO DE ENTREGA

ServiNow

Aplicación móvil de solicitudes y ofertas de servicios

SEGUNDA FASE

Implementación funcional y evidencias

Santiago Dorado
Juan David Saza
Fabián López

Kotlin · Jetpack Compose · MVVM · Hilt
Datos de evaluación en memoria

Repositorio de código fuente

https://github.com/JDFaFa/ServiNow

Capturas obtenidas de la aplicación ejecutada en un emulador Android. No se utilizaron imágenes del prototipo como prueba de implementación.

GUÍA DE LECTURA

Contenido y trazabilidad

Alcance y evaluación · páginas 3-4

Incluye el enlace a la versión exacta del código, las exenciones de esta fase, las cuentas de prueba y el recorrido recomendado.

Requisitos funcionales · páginas 5-47

Una ficha con captura por cada requisito original RF-01 a RF-43. Se conserva su identificación y se explican las adaptaciones autorizadas.

Requisitos técnicos · páginas 48-54

RNF-01 a RNF-07: plataforma, Kotlin/Compose, almacenamiento, imágenes, autenticación, avisos y presentación. Los requisitos exentos se declaran como tales.

Evidencias adicionales · páginas 55-72

EV-01 a EV-18: ícono, logo, onboarding, errores y confirmaciones de formularios, ofertas, seguimiento, estadísticas y estados vacíos.

Arquitectura y cambios · páginas 73-74

MVVM, Hilt y Navigation Compose, reglas de negocio, diferencias frente al prototipo y razones de los cambios.

Uso de IA y verificación · páginas 75-79

Herramientas, propósito, prompts relevantes, dificultades, correcciones, pruebas realizadas y referencias.

Cómo interpretar una ficha

La captura evidencia un estado concreto de la app; el texto explica su alcance. Las pruebas y el código complementan lo que una imagen no puede demostrar. EV amplía la evidencia sin alterar la numeración RF/RNF original.

01 · ALCANCE

Qué incluye esta entrega

Aplicación funcional

Solicitudes, ofertas, comentarios, calificaciones, perfiles, currículum, avisos, estadísticas, reputación, logros y moderación se conectan mediante un repositorio compartido en memoria. Los cambios de un usuario son visibles al cambiar a otro usuario de prueba dentro del mismo proceso.

Adaptaciones autorizadas por la consigna

No se exige base de datos, sesiones persistentes, gestión de permisos, mapas ni carga externa de imágenes. Por ello se usan cuentas y datos de prueba en memoria, imágenes ilustrativas incluidas con la app y ciudades de referencia. El mapa de búsqueda existente se conserva como complemento opcional.

Simulaciones identificadas

La recuperación utiliza un enlace de prueba dentro de la app: modifica la clave en memoria y no envía correos. El botón Google informa que requiere un proveedor externo; no se presenta como OAuth operativo. Las cuentas de moderador están precargadas en el repositorio, no en una base de datos.

Alcance de la evidencia

Cada ficha vincula un requisito con una captura real y describe la acción, resultado y adaptación respecto al prototipo. Los datos de personas y trabajos que aparecen son ficticios. Los requisitos técnicos exentos se identifican expresamente en el anexo.

Versión de código

Código evaluado y publicado: 5d4a874 (rama main). Abrir la versión exacta en GitHub. Se comprobó el identificador remoto el 17 de octubre de 2026. El PDF actualizado se entrega por separado del commit de código.

02 · EVALUACIÓN

Cómo probar la aplicación

Cuentas de ejemplo

Solicitante: valentina@gmail.com / ServiNow1!
Trabajador: andres@gmail.com / ServiNow1!
Otro usuario: laura@hotmail.com / ServiNow1!
Moderador: moderador@gmail.com / Moderador1!
Son claves públicas de demostración, sin relación con cuentas reales.

Recorrido principal

1. Entrar como Valentina y crear una solicitud.
2. Cambiar al moderador y verificarla o rechazarla con motivo.
3. Entrar como Andrés, ofertar, comentar y marcar interés.
4. Volver a Valentina: leer avisos, aceptar una oferta, finalizar y reseñar.
5. Consultar el perfil del trabajador, estadísticas y logros.

Registro y recuperación

Registrar un correo nuevo permitido, iniciar sesión con él y probar una clave incorrecta. Solicitar la recuperación, abrir el enlace simulado y establecer una clave nueva. El cambio se aplica al usuario en memoria.

Reinicio de datos

Cerrar el proceso y abrir la app restaura las cuentas y trabajos iniciales. Cambiar de pantalla o de usuario dentro del mismo proceso conserva los cambios. Las capturas pueden reflejar datos modificados por la secuencia de pruebas.

Integrantes

Santiago Dorado, Juan David Saza y Fabián López. No se atribuye revisión manual o contribuciones de Git a un integrante sin evidencia de que las haya realizado.

RF-01 · Registro de usuario



IMPLEMENTACIÓN

Se ingresan nombre, correo y contraseña con confirmación. Se valida el dominio permitido y la complejidad de la clave; no se admiten correos duplicados.

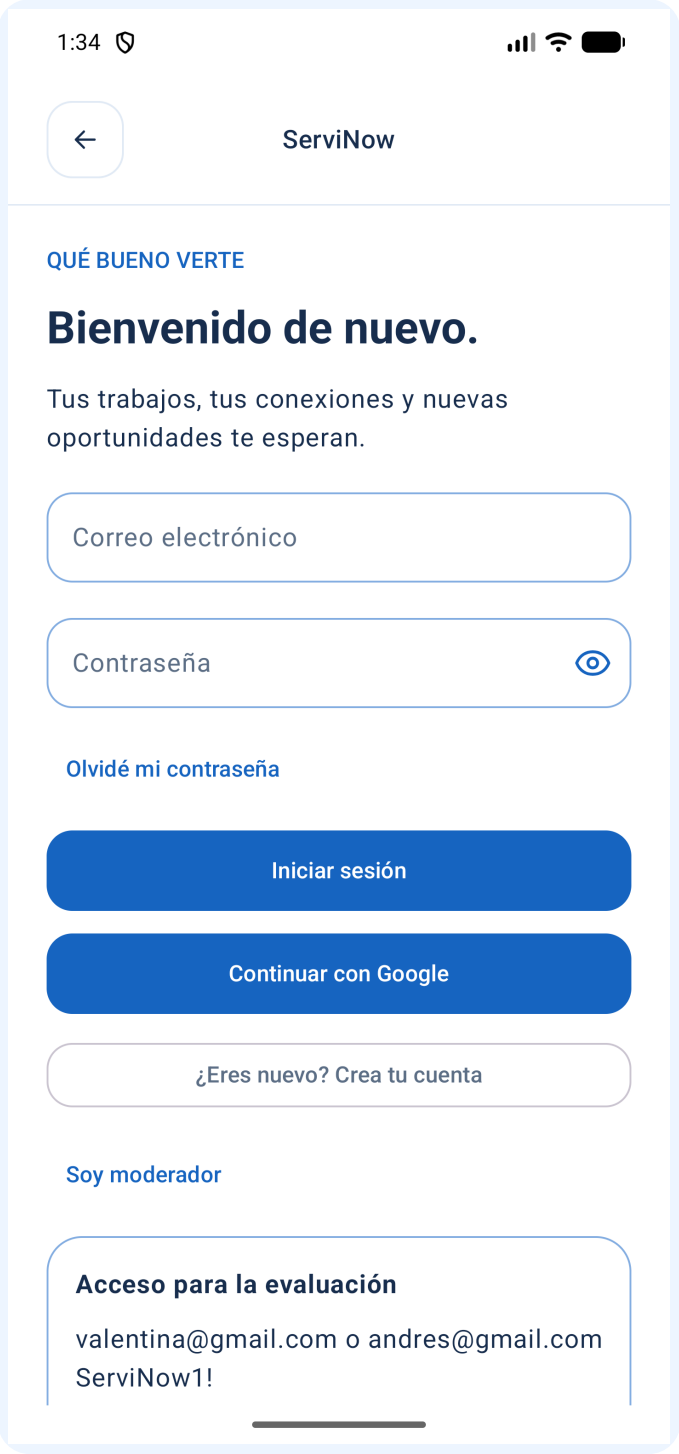
RESPECTO AL PROTOTIPO

Una sola cuenta puede solicitar y ofrecer servicios. La creación ocurre en memoria; la dirección se completa después.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 02-register.png

RF-02 · Inicio de sesión



IMPLEMENTACIÓN

El correo y la clave se comparan con las cuentas en memoria. Las credenciales incorrectas producen un diálogo; el rol determina la pantalla inicial.

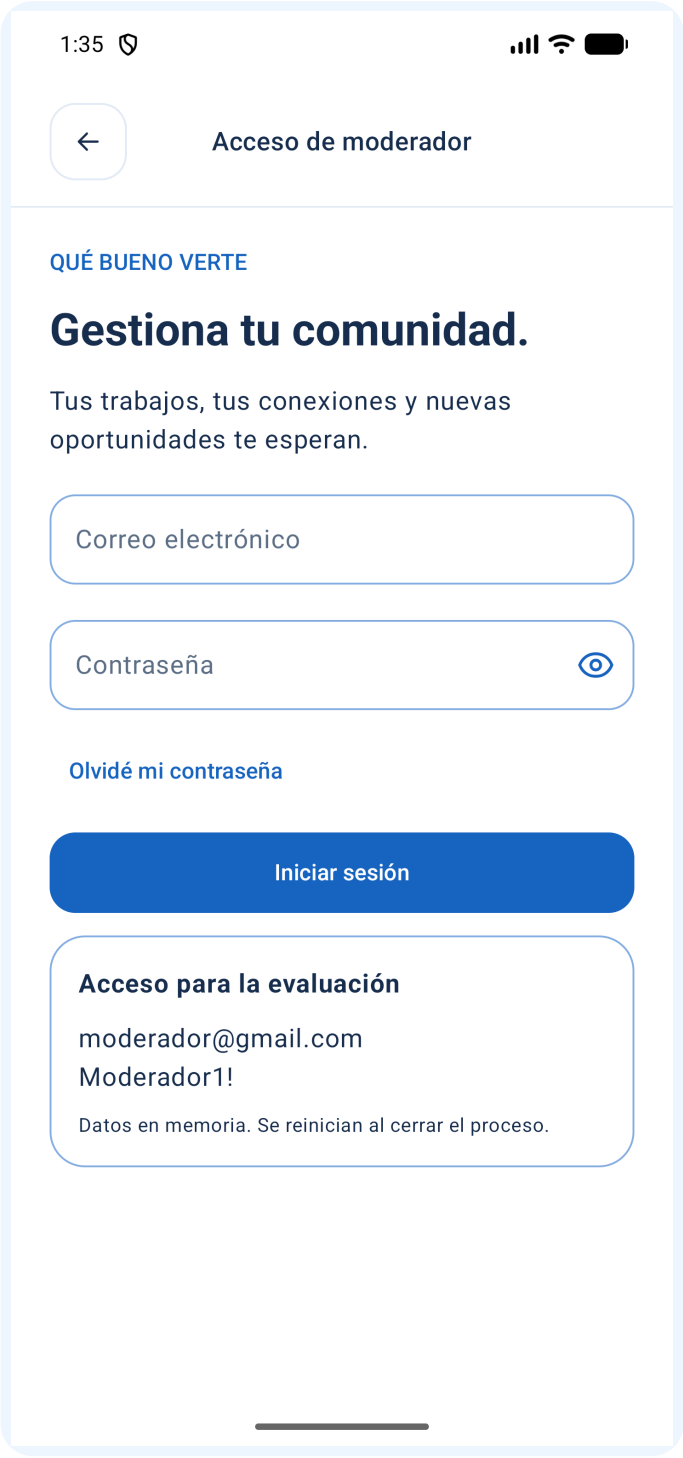
RESPECTO AL PROTOTIPO

No se mantiene una sesión entre procesos. Google queda identificado como integración externa pendiente.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 01-login.png

RF-03 · Diferenciación de roles y permisos



IMPLEMENTACIÓN

Una cuenta precargada accede a la bandeja de revisión. Las credenciales de usuario no conceden privilegios de moderación.

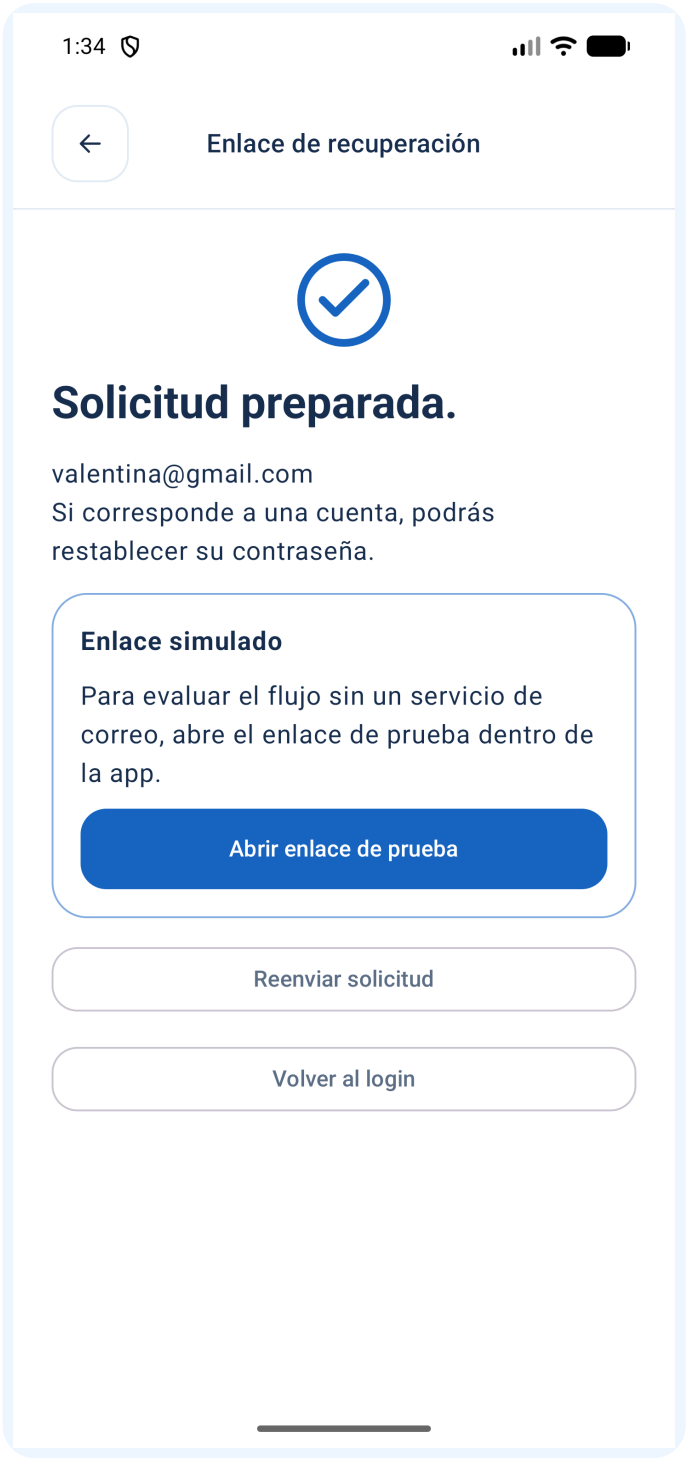
RESPECTO AL PROTOTIPO

La comprobación se hace en memoria; no representa seguridad de un servidor ni una sesión persistente.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 39-mod-login.png

RF-04 · Recuperación de contraseña



IMPLEMENTACIÓN

Solicitar recuperación abre la confirmación y un enlace de prueba hacia Nueva contraseña. El cambio se valida y modifica la clave en memoria. Las evidencias adicionales muestran la confirmación final.

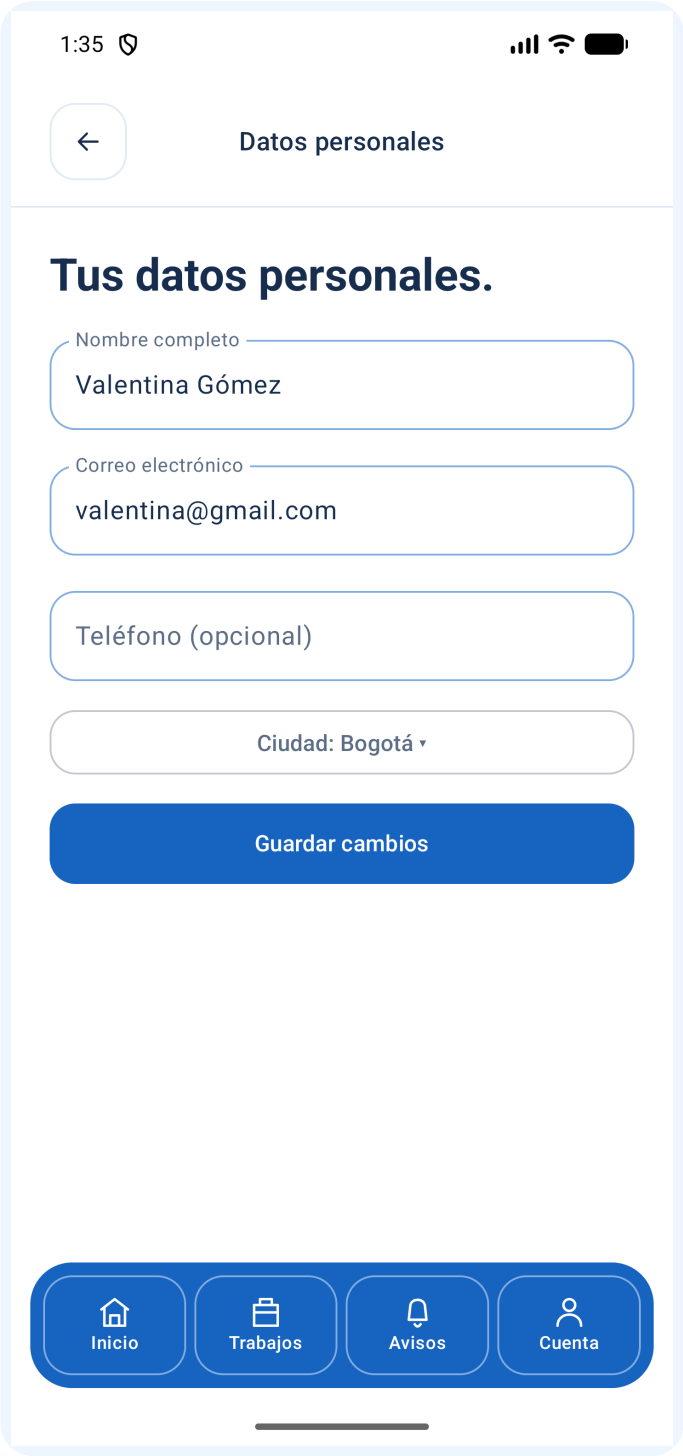
RESPECTO AL PROTOTIPO

Adaptación de fase: no se envía correo real. El enlace está explícitamente simulado; no se presenta el envío de correo del requisito original como implementado.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 06-sent.png

RF-05 · Edición de datos personales



IMPLEMENTACIÓN

El usuario cambia nombre, correo, teléfono y ciudad. Se valida el correo y se impiden duplicados; no puede cambiar su rol.

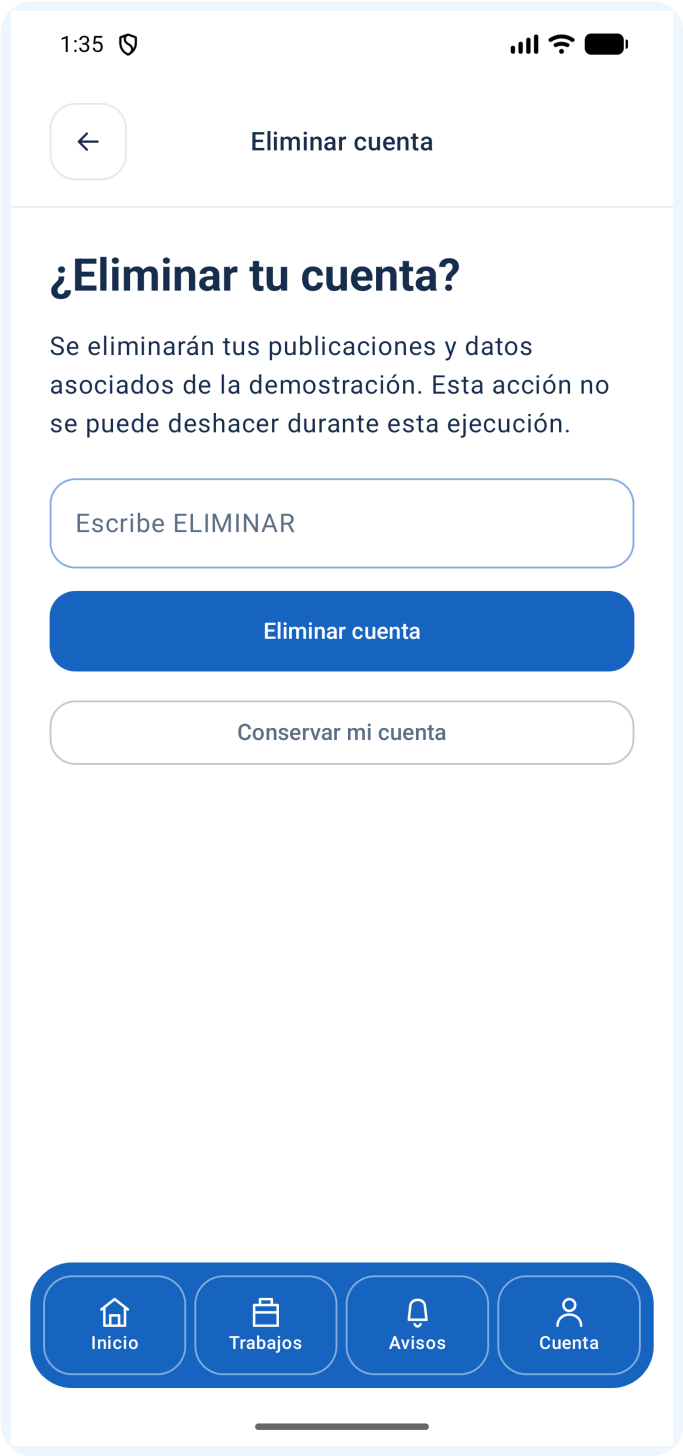
RESPECTO AL PROTOTIPO

La dirección se simplifica a ciudad en esta fase, que no exige mapa ni permisos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 37-edit-account.png

RF-06 · Eliminación de cuenta



IMPLEMENTACIÓN

Se requiere escribir ELIMINAR. Se retiran el usuario y sus datos asociados de la demostración y se vuelve al acceso.

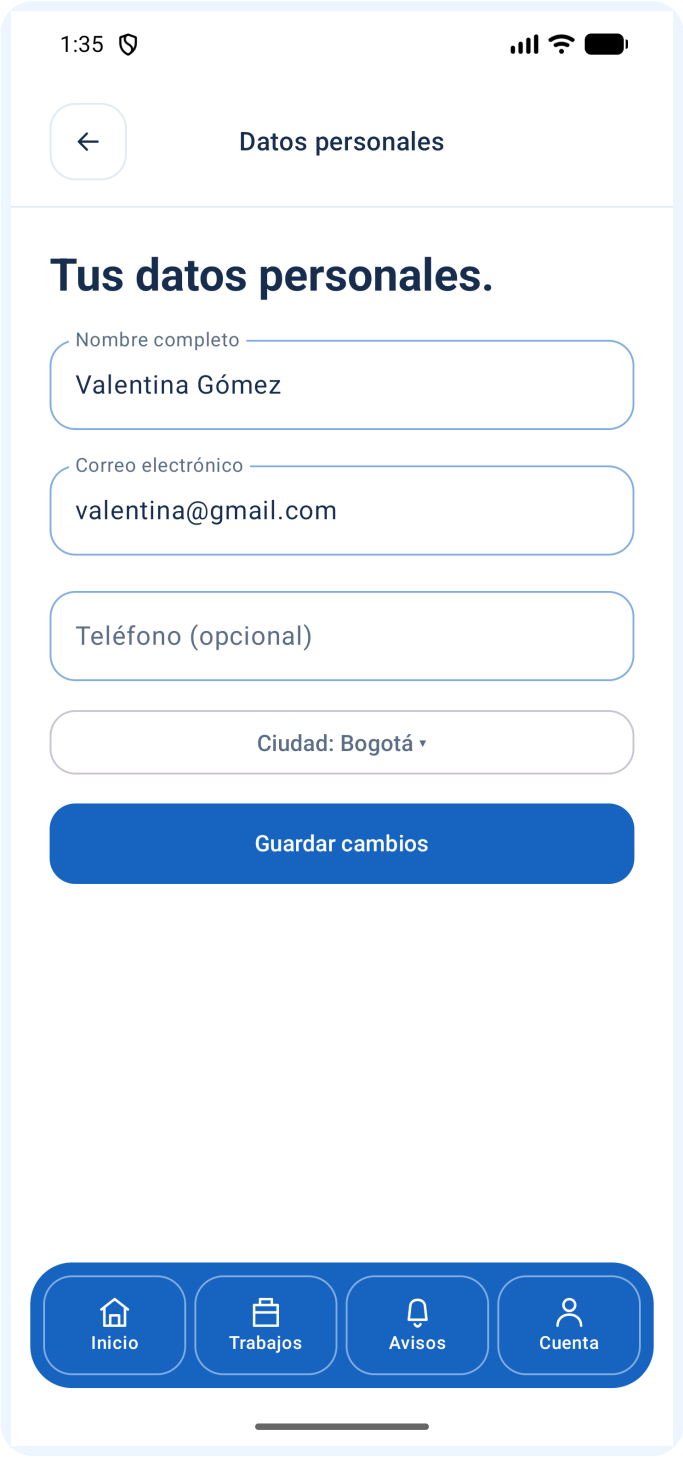
RESPECTO AL PROTOTIPO

Las cuentas precargadas de moderador no se eliminan desde el menú de usuario.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 38-delete-account.png

RF-07 · Selección de la dirección del usuario



IMPLEMENTACIÓN

La edición de datos conserva una ciudad de referencia. No almacena coordenadas personales ni una dirección obtenida de mapa.

RESPECTO AL PROTOTIPO

EXENTO EN FASE 2: mapas y base de datos. La captura muestra la alternativa de ciudad; no constituye evidencia de geolocalización personal completa.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 37-edit-account.png

RF-08 · Consulta del feed de publicaciones



IMPLEMENTACIÓN

El inicio presenta el acceso a publicar, buscar trabajo y consultar el feed público. Se puede abrir el mapa opcional para fijar el centro de búsqueda.

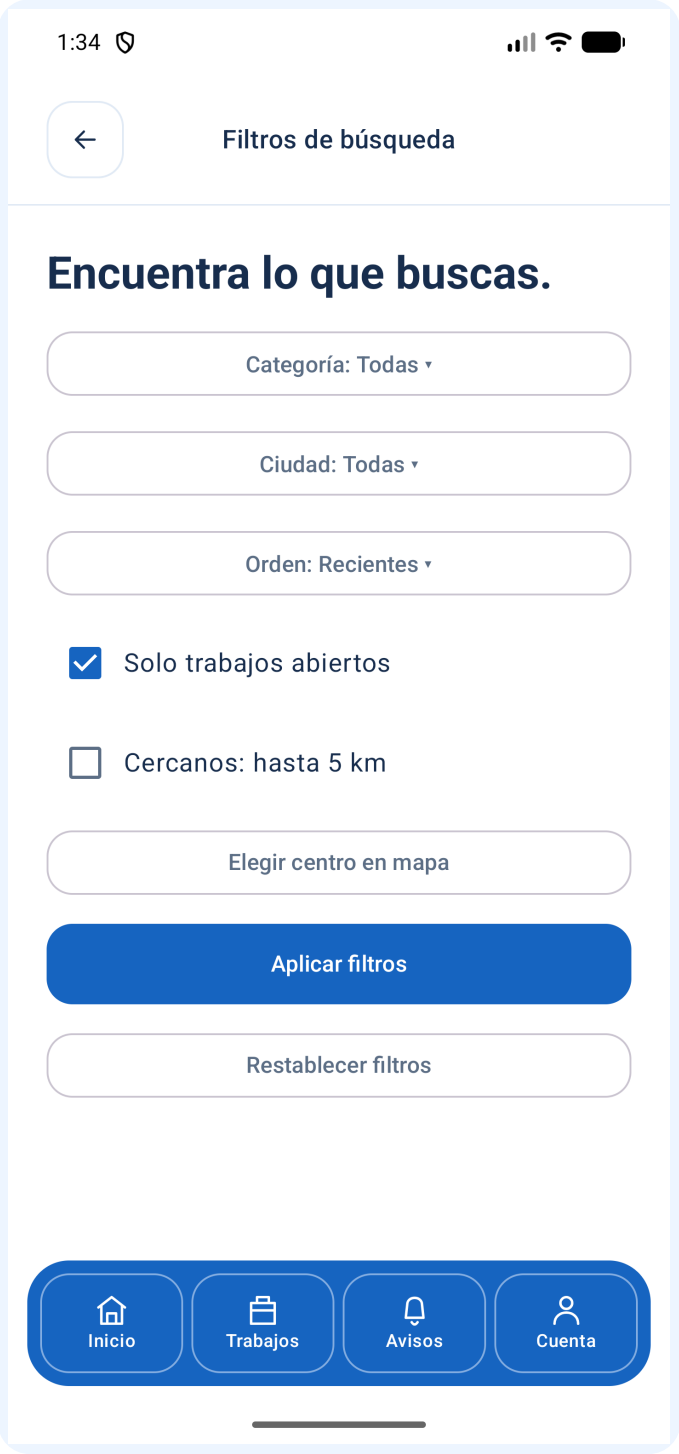
RESPECTO AL PROTOTIPO

El feed implementado es una lista. El mapa selecciona una zona; no es un mapa de marcadores de todas las publicaciones. La modalidad mapa está exenta en esta fase.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 08-home.png

RF-09 · Filtrado por categoría



IMPLEMENTACIÓN

La selección de categoría filtra la lista de trabajos; Todas elimina esa restricción.

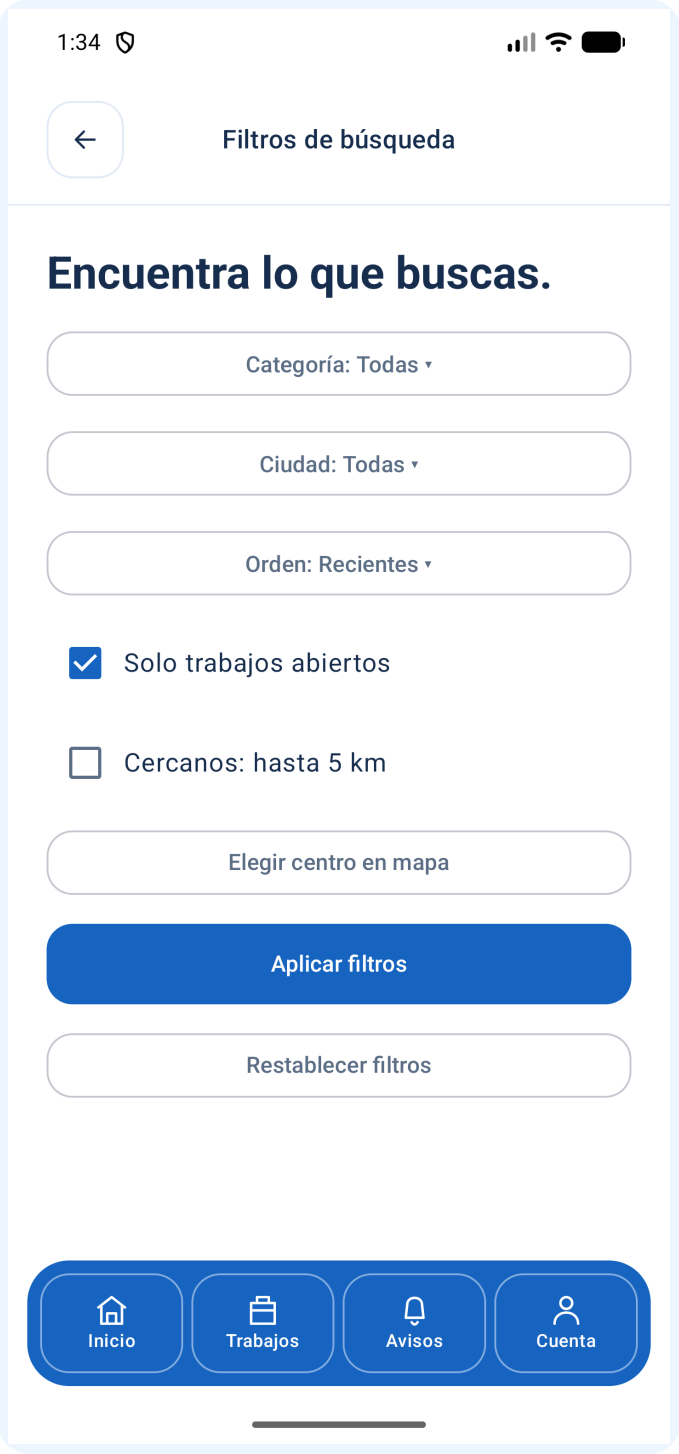
RESPECTO AL PROTOTIPO

Los filtros funcionan con las coordenadas de los datos de prueba. La cercanía usa una distancia de hasta 5 km.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 09-filters.png

RF-10 · Filtrado por ubicación



IMPLEMENTACIÓN

Permite filtrar por ciudad y por cercanía al punto elegido. El grupo fijó un radio de 5 km; el cálculo usa las coordenadas de los trabajos de ejemplo.

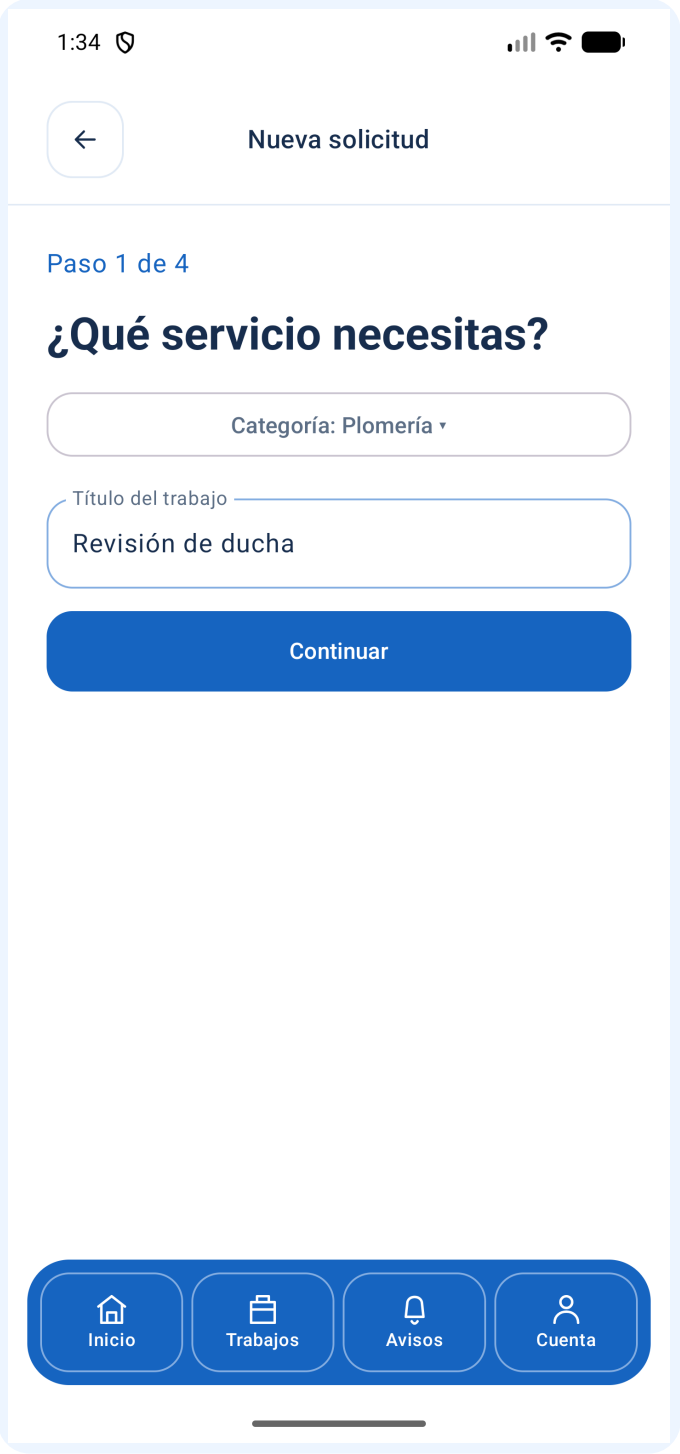
RESPECTO AL PROTOTIPO

Los filtros funcionan con las coordenadas de los datos de prueba. La cercanía usa una distancia de hasta 5 km.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 09-filters.png

RF-11 · Creación de una publicación de trabajo



IMPLEMENTACIÓN

El usuario selecciona categoría y título. Los pasos siguientes recogen descripción, presupuesto y ubicación de referencia.

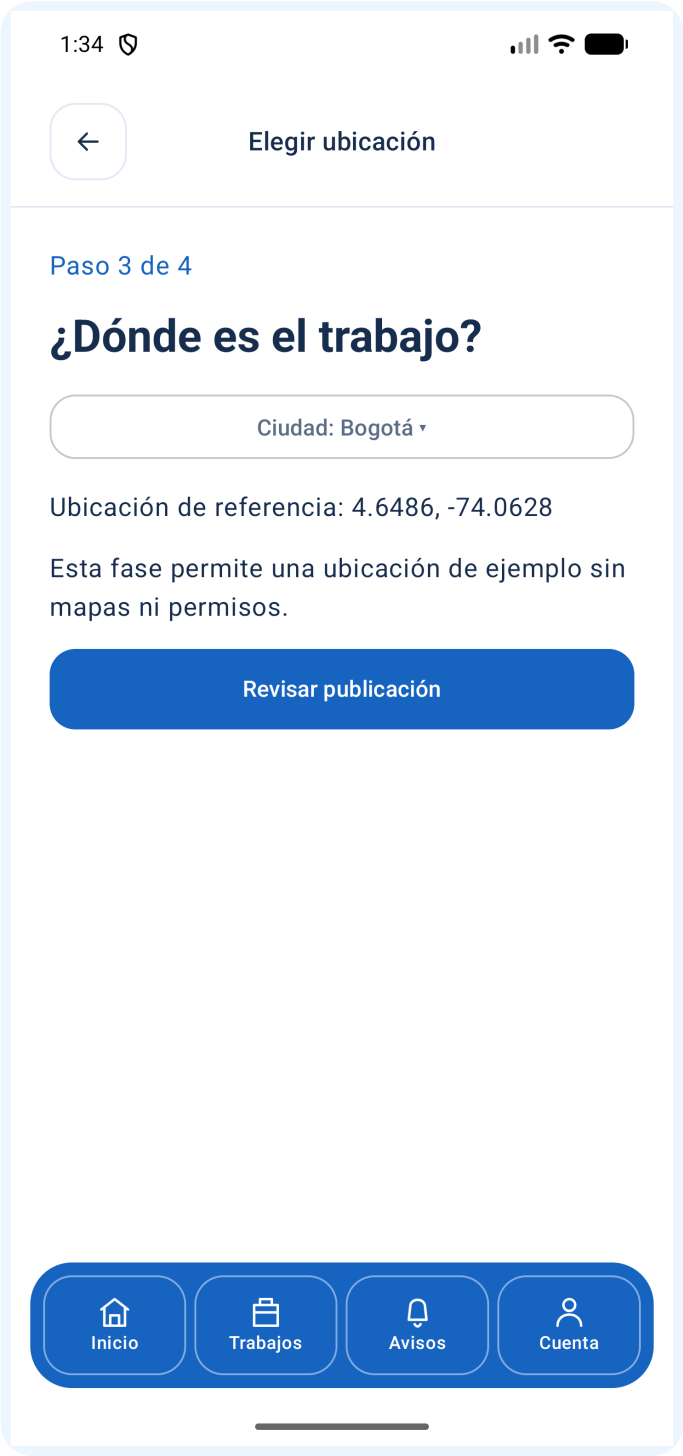
RESPECTO AL PROTOTIPO

Se conserva el formulario por pasos y se añaden validaciones antes de avanzar.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 13-create.png

RF-12 · Selección de la ubicación del trabajo



IMPLEMENTACIÓN

La ciudad seleccionada asigna coordenadas de referencia al trabajo.

RESPECTO AL PROTOTIPO

Se evita exigir permisos o un mapa en el registro de publicaciones, según la consigna de la fase dos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 15-post-location.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-13 · Incorporación de imágenes a la publicación



IMPLEMENTACIÓN

La descripción y el presupuesto se validan. Se selecciona una imagen ilustrativa para poder revisar la publicación completa.

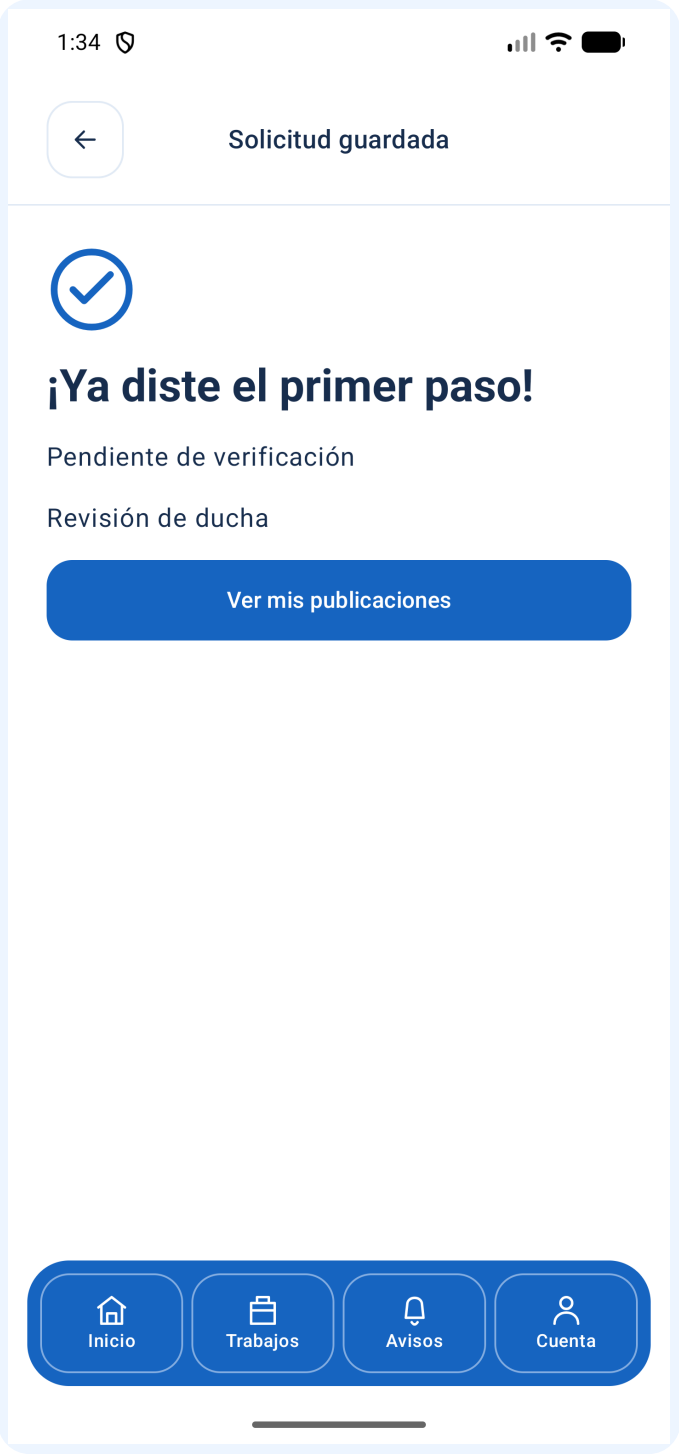
RESPECTO AL PROTOTIPO

La carga de imágenes está exenta. El botón de ejemplo no sube archivos ni simula un servicio de almacenamiento.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 14-create-details.png

RF-14 · Estado inicial sin verificar



IMPLEMENTACIÓN

Al guardar, la publicación queda pendiente, genera un aviso y no se hace pública hasta la verificación.

RESPECTO AL PROTOTIPO

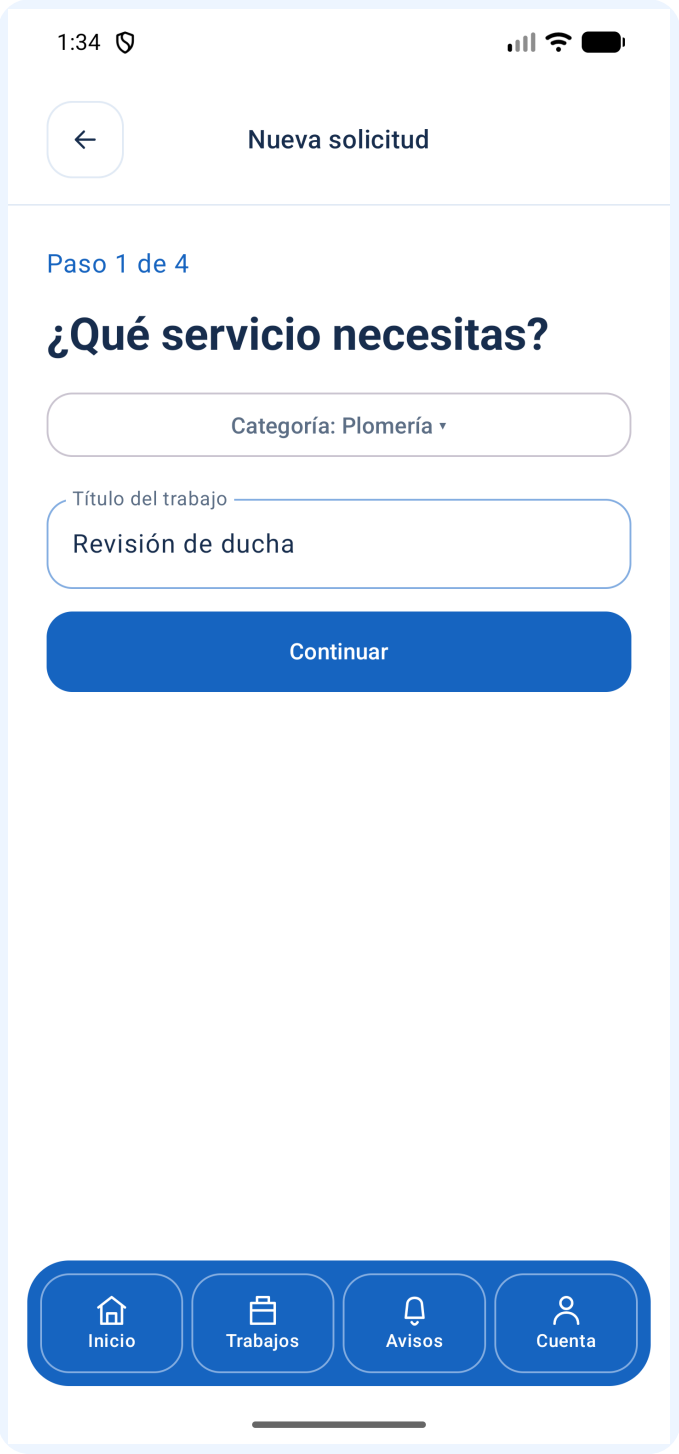
El cambio de estado es funcional en memoria y puede comprobarse con la cuenta de moderador.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 17-submitted.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-15 · Edición de publicaciones propias



IMPLEMENTACIÓN

El autor puede editar trabajos pendientes, verificados o rechazados. Guardar una edición devuelve la solicitud a revisión.

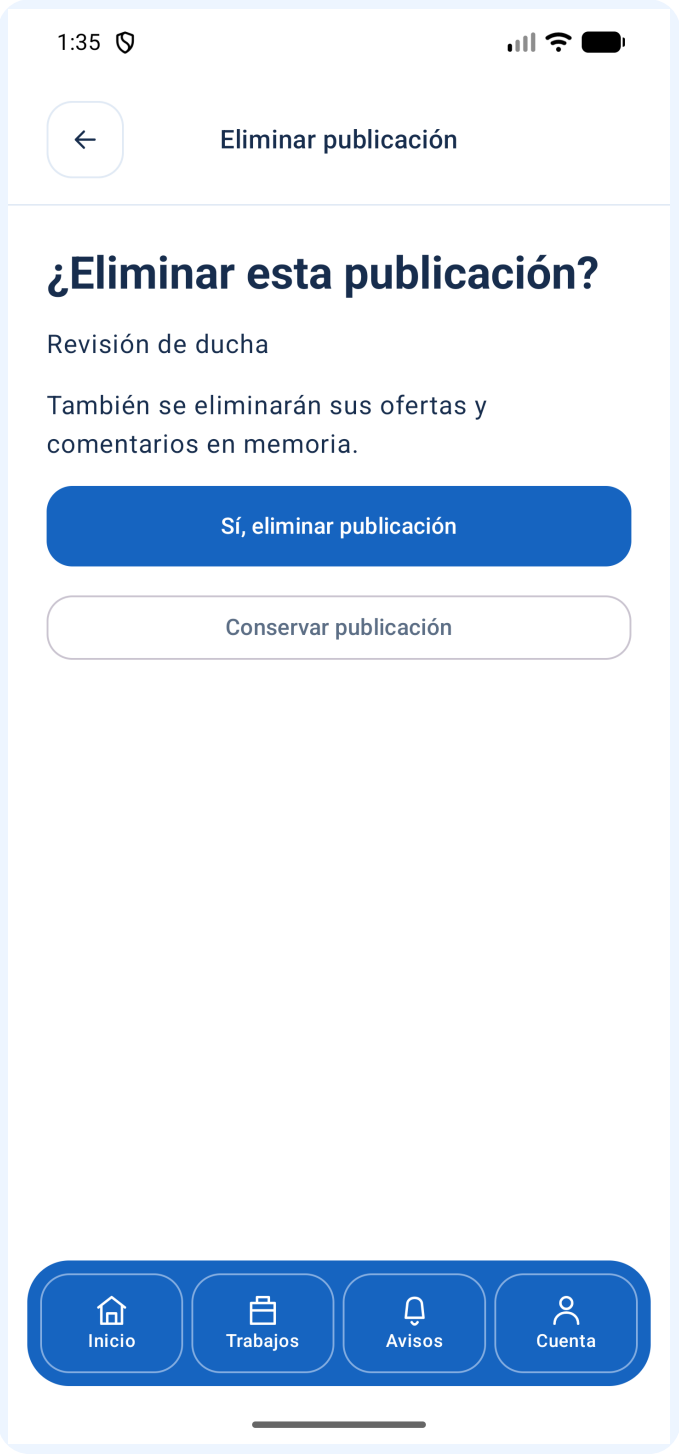
RESPECTO AL PROTOTIPO

Los trabajos asignados o finalizados no se editan para proteger el acuerdo existente.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 19-edit.png

RF-16 · Eliminación de publicaciones propias



IMPLEMENTACIÓN

Solo el autor puede eliminar. La pantalla confirma la intención y se eliminan también sus ofertas, comentarios y reseñas asociadas.

RESPECTO AL PROTOTIPO

Se aplica al repositorio en memoria; no afecta archivos originales del dispositivo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 26-delete-post.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-17 · Consulta del detalle de una publicación



IMPLEMENTACIÓN

Presenta título, categoría, estado, descripción, presupuesto, ciudad, coordenadas, imagen de ejemplo y autor.

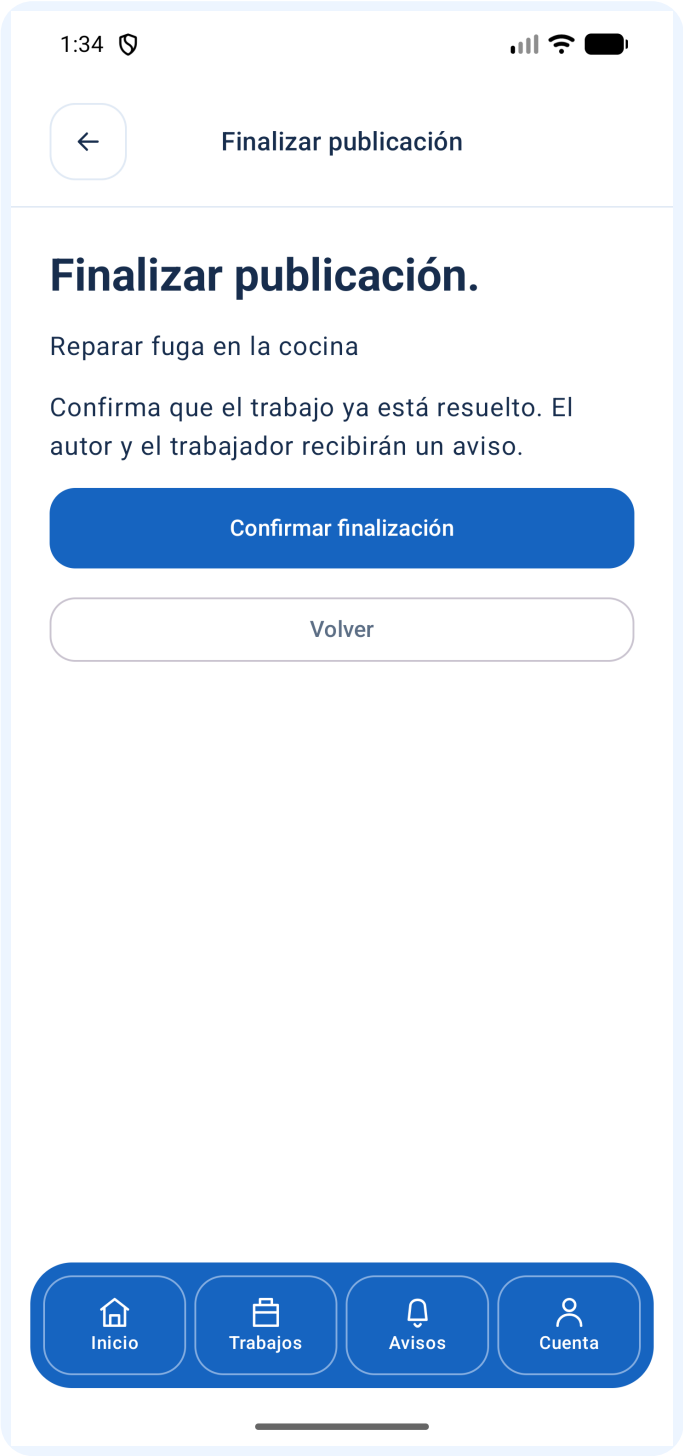
RESPECTO AL PROTOTIPO

Las fotos remotas se sustituyen por una ilustración identificada como ejemplo; el detalle conserva la información requerida.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 11-detail.png

RF-18 · Finalización por el autor



IMPLEMENTACIÓN

El autor puede confirmar que su publicación está resuelta. Se actualiza el estado y se avisa al autor y al trabajador asignado.

RESPECTO AL PROTOTIPO

La finalización habilita la reseña cuando existe un trabajador asignado.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 24-finish.png

RF-19 · Votación de publicaciones



IMPLEMENTACIÓN

El botón Me interesa registra o retira el voto del usuario y actualiza el contador. No permite votar publicaciones propias ni otorgar puntos repetidos al alternar el voto.

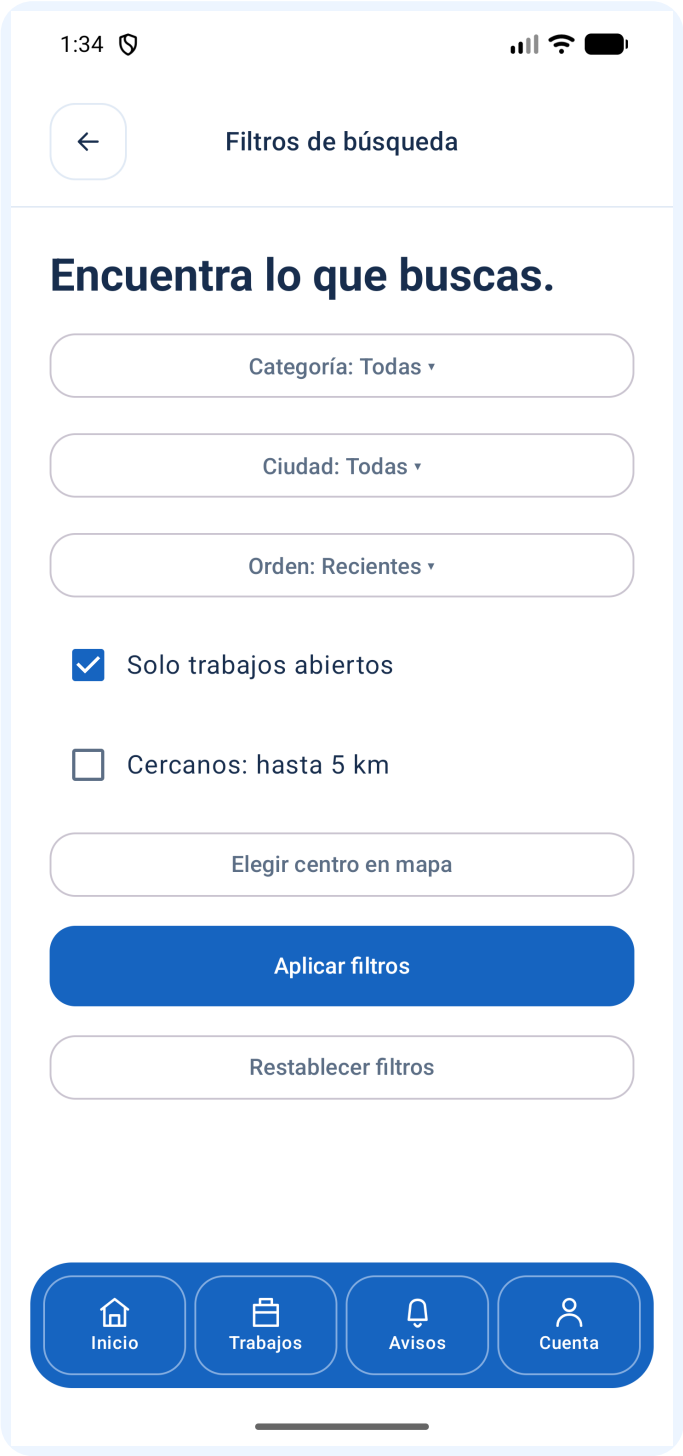
RESPECTO AL PROTOTIPO

La interacción utiliza el estado compartido en memoria. Las pruebas unitarias verifican la prevención de puntos duplicados.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 11-detail.png

RF-20 · Determinación de relevancia



IMPLEMENTACIÓN

La opción Más relevantes ordena por el número de usuarios que marcaron Me interesa, de mayor a menor. Cada usuario tiene un voto activo por publicación.

RESPECTO AL PROTOTIPO

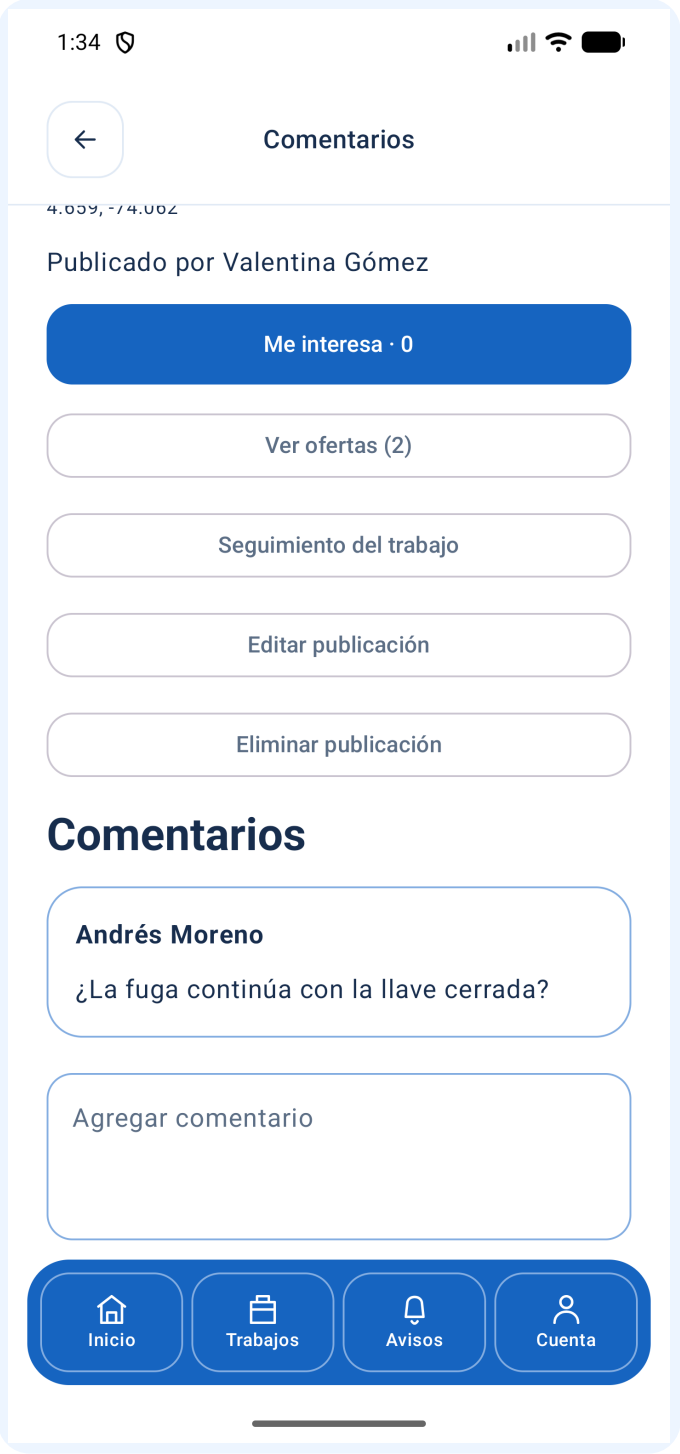
Fórmula elegida: relevancia = cantidad de votos activos. En empates se mantiene el orden de la lista de datos. No se añaden factores de reputación ni distancia.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 09-filters.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-21 · Creación de comentarios en publicaciones



IMPLEMENTACIÓN

Los comentarios se agregan a la publicación, indican autor y quedan visibles al cambiar de usuario. Un comentario ajeno genera un aviso al autor.

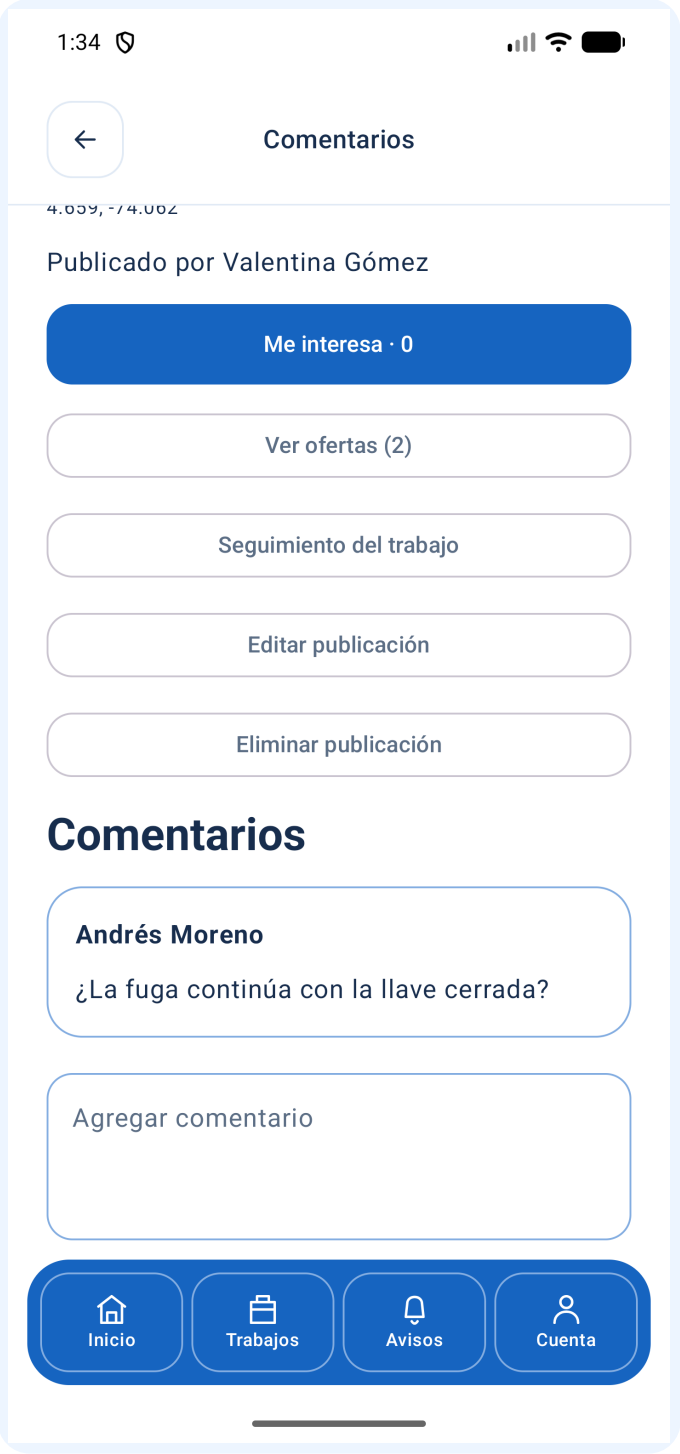
RESPECTO AL PROTOTIPO

No se usa chat externo ni entrega en tiempo real. Los comentarios se guardan en el repositorio en memoria.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 12-comments.png

RF-22 · Consulta de comentarios de una publicación



IMPLEMENTACIÓN

Los comentarios se agregan a la publicación, indican autor y quedan visibles al cambiar de usuario. Un comentario ajeno genera un aviso al autor.

RESPECTO AL PROTOTIPO

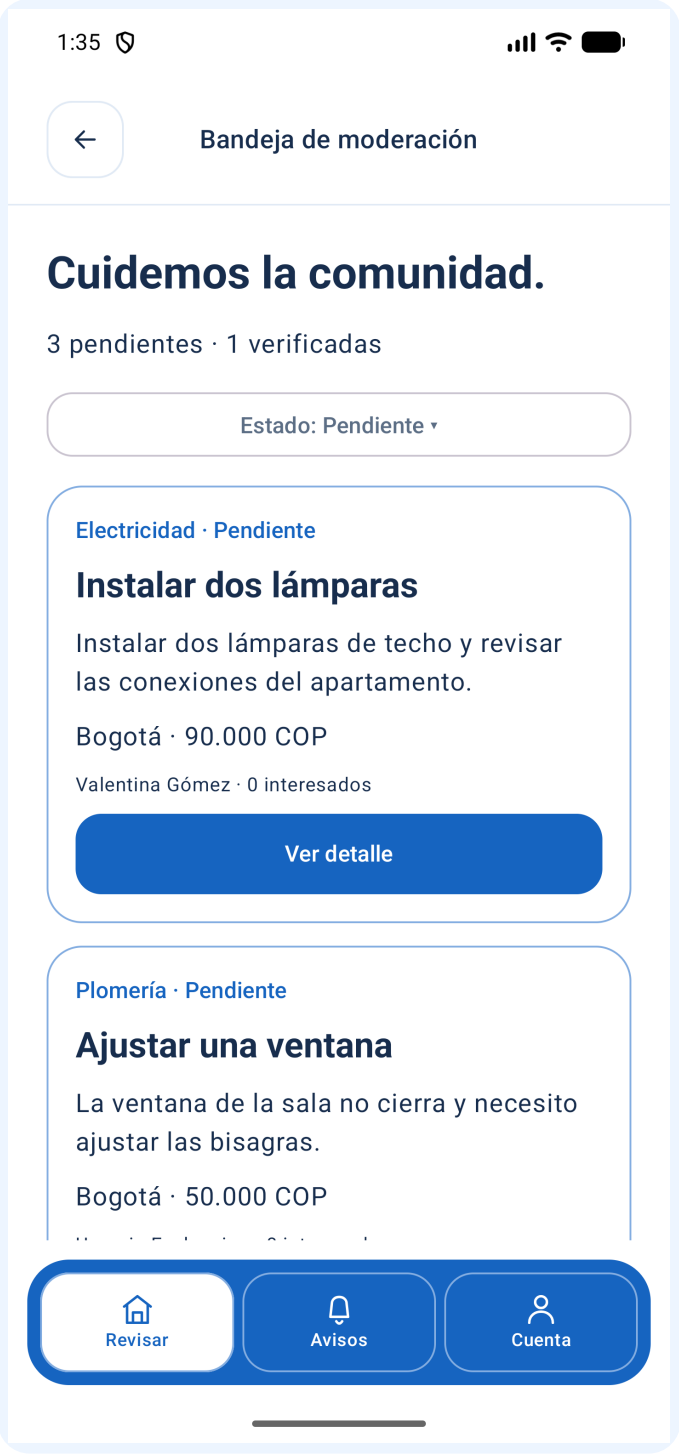
No se usa chat externo ni entrega en tiempo real. Los comentarios se guardan en el repositorio en memoria.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 12-comments.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-23 · Consulta de publicaciones para moderación



IMPLEMENTACIÓN

La bandeja del moderador permite listar publicaciones y filtrar por estado antes de abrir el detalle de revisión.

RESPECTO AL PROTOTIPO

Se sustituyeron los botones informativos de la etapa anterior por operaciones del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 40-moderation.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-24 · Verificación de publicaciones



IMPLEMENTACIÓN

La bandeja permite filtrar por estado e inspeccionar una publicación. Verificarla la vuelve pública, otorga puntos y avisa al autor.

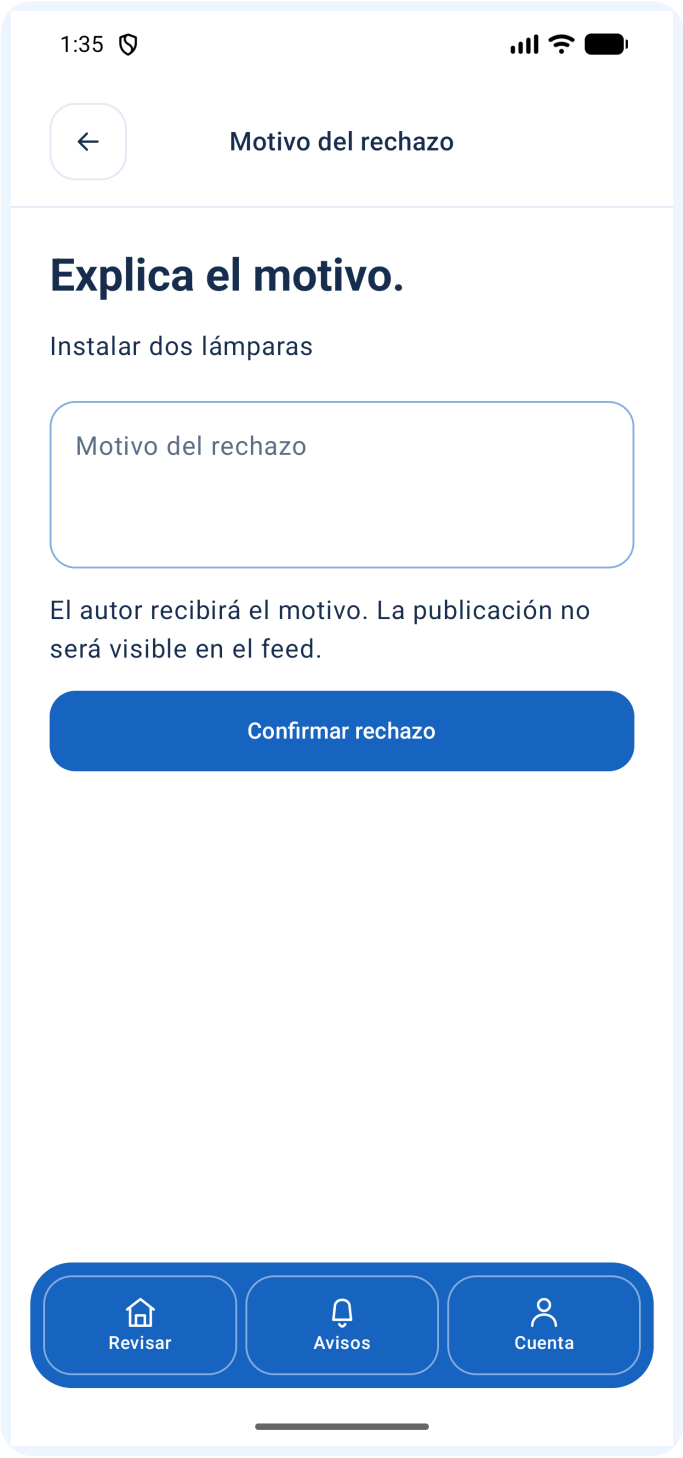
RESPECTO AL PROTOTIPO

Se sustituyeron los botones informativos de la etapa anterior por operaciones del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 41-mod-detail.png

RF-25 · Rechazo con motivo obligatorio



IMPLEMENTACIÓN

Se exige un motivo de 5 a 500 caracteres. Se cambia a rechazada, se excluye del feed y se notifica al autor.

RESPECTO AL PROTOTIPO

La validación se ejecuta en el repositorio, incluso si se intenta invocar la acción sin completar el formulario.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 42-reject.png

RF-26 · Exclusión de publicaciones rechazadas



IMPLEMENTACIÓN

El autor puede ver el motivo de rechazo en su detalle. El feed público filtra únicamente estados verificada, en progreso y finalizada; no incluye rechazadas.

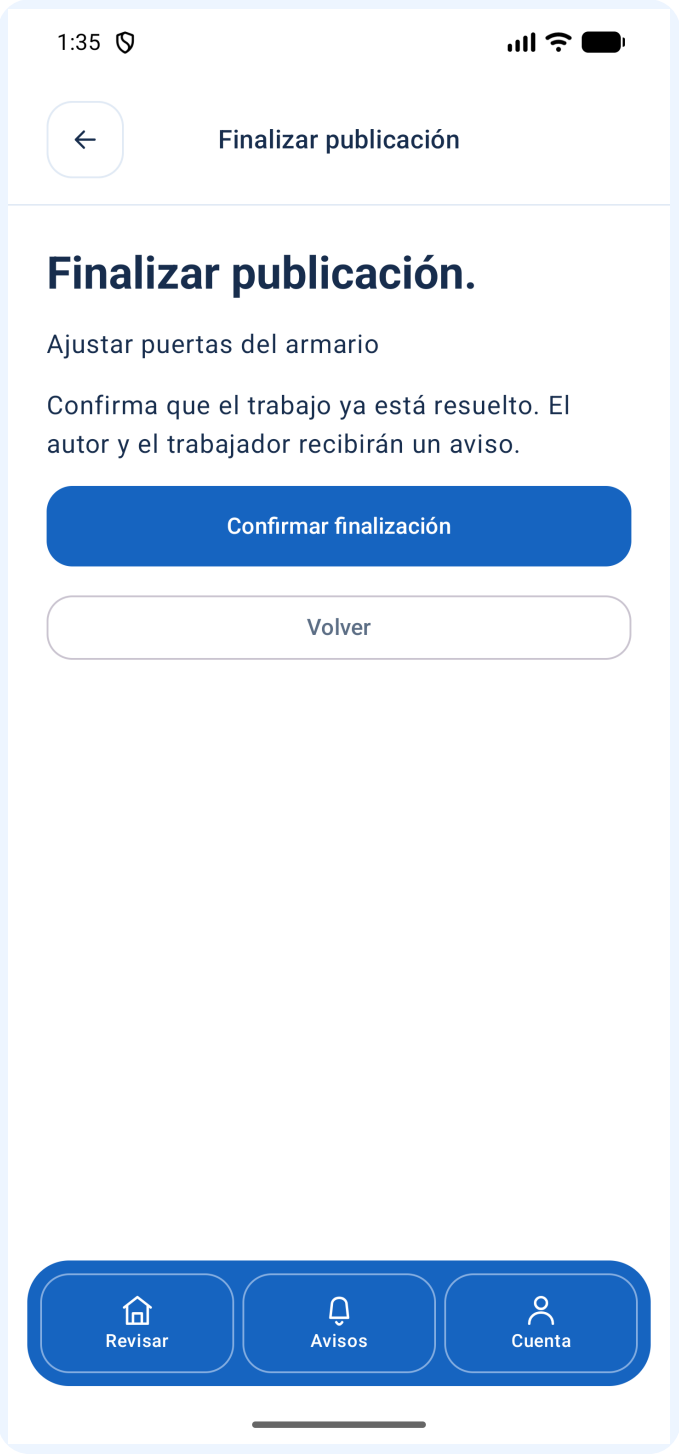
RESPECTO AL PROTOTIPO

La captura muestra el estado rechazado para su autor. La exclusión se sustenta además en el filtro del código y las pruebas; una sola captura no demuestra la ausencia en todos los listados.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 20-rejected.png

RF-27 · Finalización por el moderador



IMPLEMENTACIÓN

El moderador puede finalizar cualquier publicación aún no finalizada, con una confirmación previa.

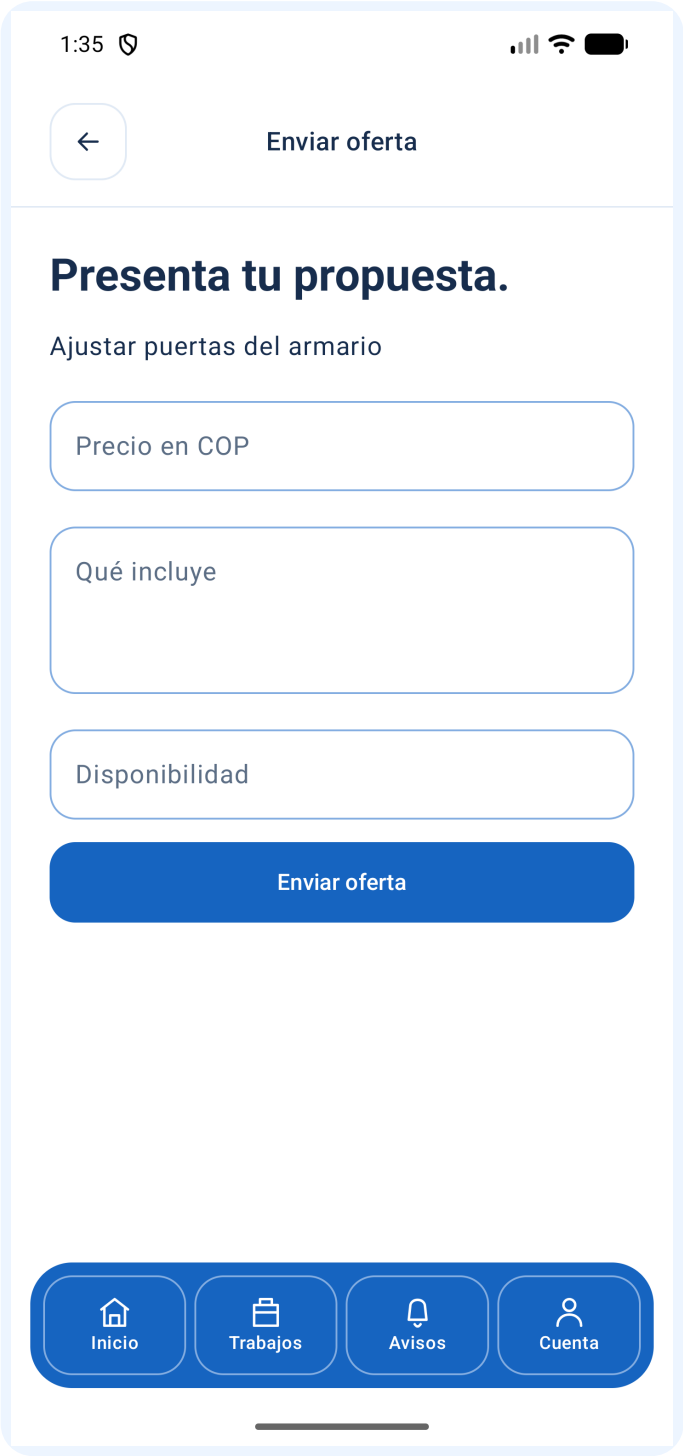
RESPECTO AL PROTOTIPO

La comprobación de rol se conserva en la operación de datos, además del acceso de la interfaz.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 43-mod-finish.png

RF-28 · Presentación de ofertas de trabajo



IMPLEMENTACIÓN

El trabajador introduce precio, alcance y disponibilidad. Solo se aceptan ofertas en trabajos verificados, ajenos y sin una oferta previa del mismo usuario.

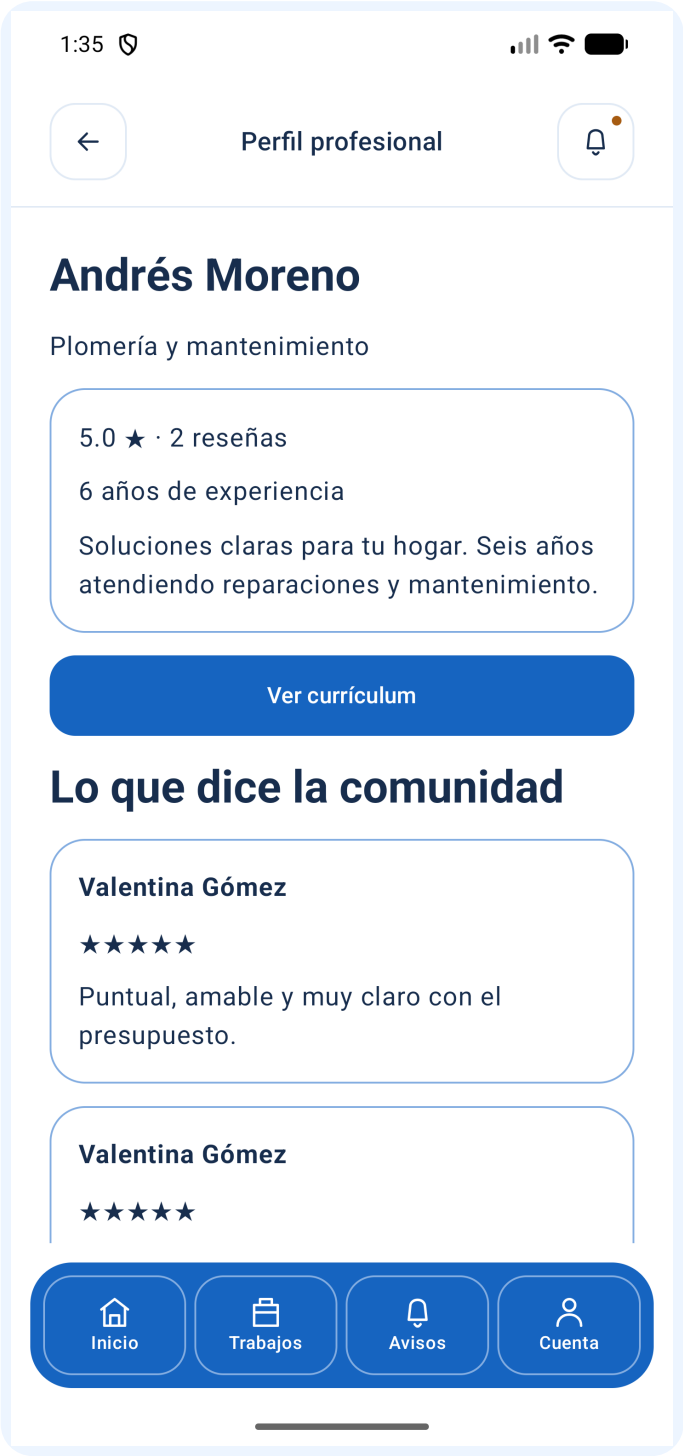
RESPECTO AL PROTOTIPO

La oferta se guarda en memoria y genera un aviso al solicitante.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 28-offer.png

RF-29 · Consulta del perfil del trabajador



IMPLEMENTACIÓN

Muestra nombre, especialidad, presentación, experiencia, calificaciones y reseñas de la comunidad.

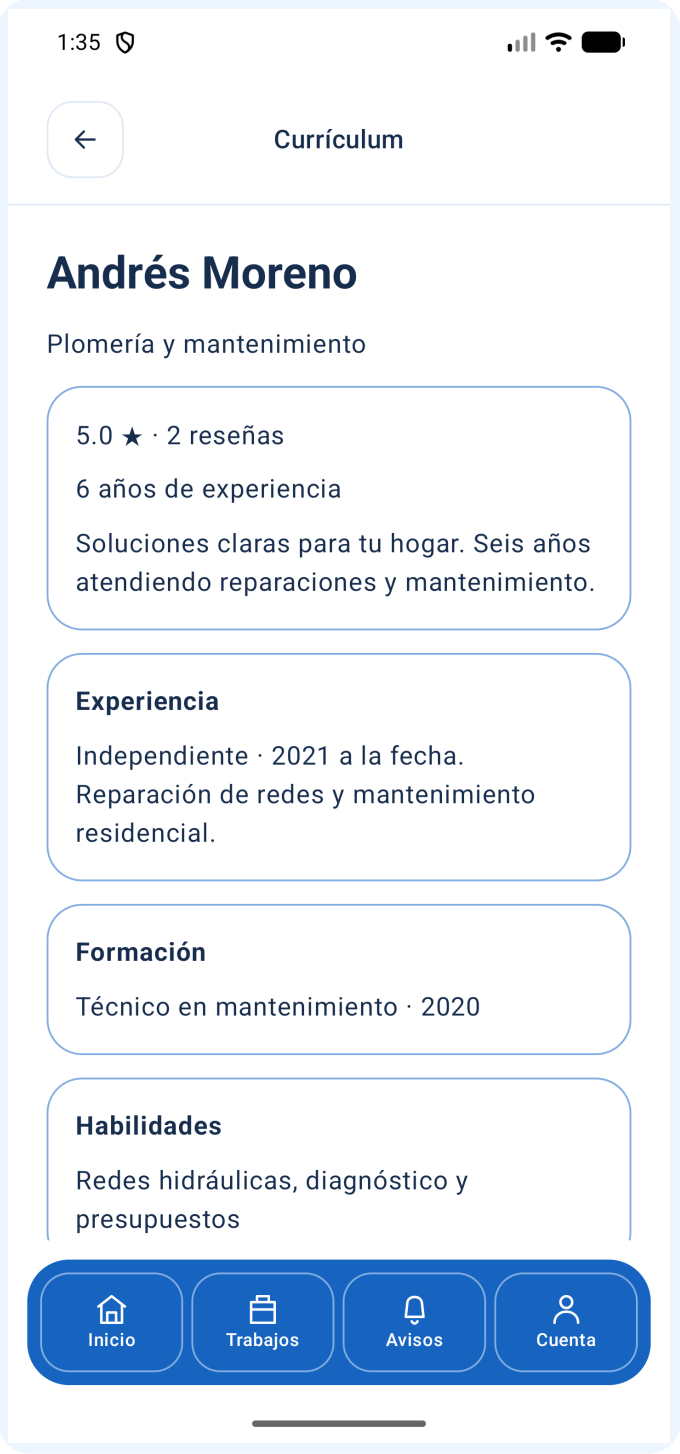
RESPECTO AL PROTOTIPO

Se conserva una tarjeta por bloque y el encabezado solo muestra campana en perfil/cuenta.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 30-profile.png

RF-30 · Consulta del currículum del trabajador



IMPLEMENTACIÓN

El currículum presenta experiencia laboral, formación y habilidades del trabajador.

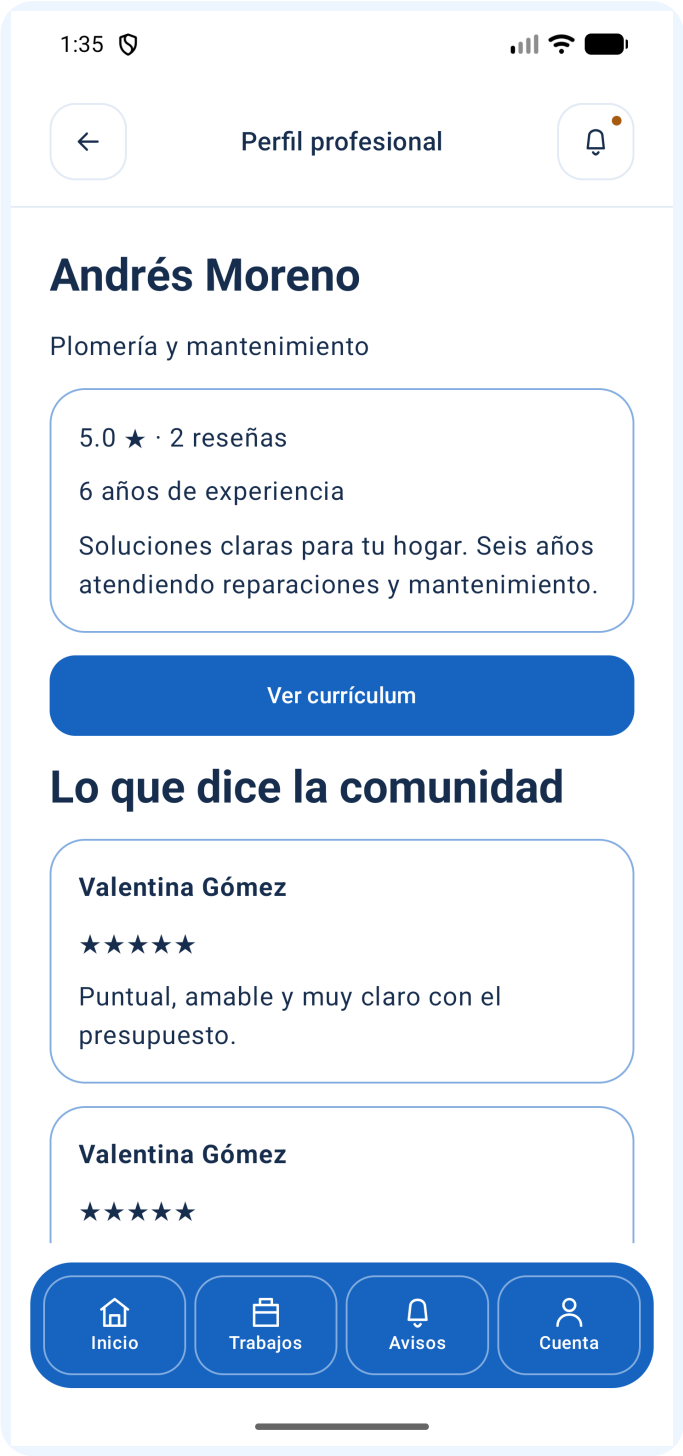
RESPECTO AL PROTOTIPO

Se genera con datos editables del perfil. No se exige subir un PDF ni gestionar permisos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 31-cv.png

RF-31 · Comentarios sobre trabajadores



IMPLEMENTACIÓN

El perfil muestra reseñas de trabajos finalizados. Para escribir una, el autor del trabajo debe haber contratado al trabajador y no haber reseñado antes esa publicación.

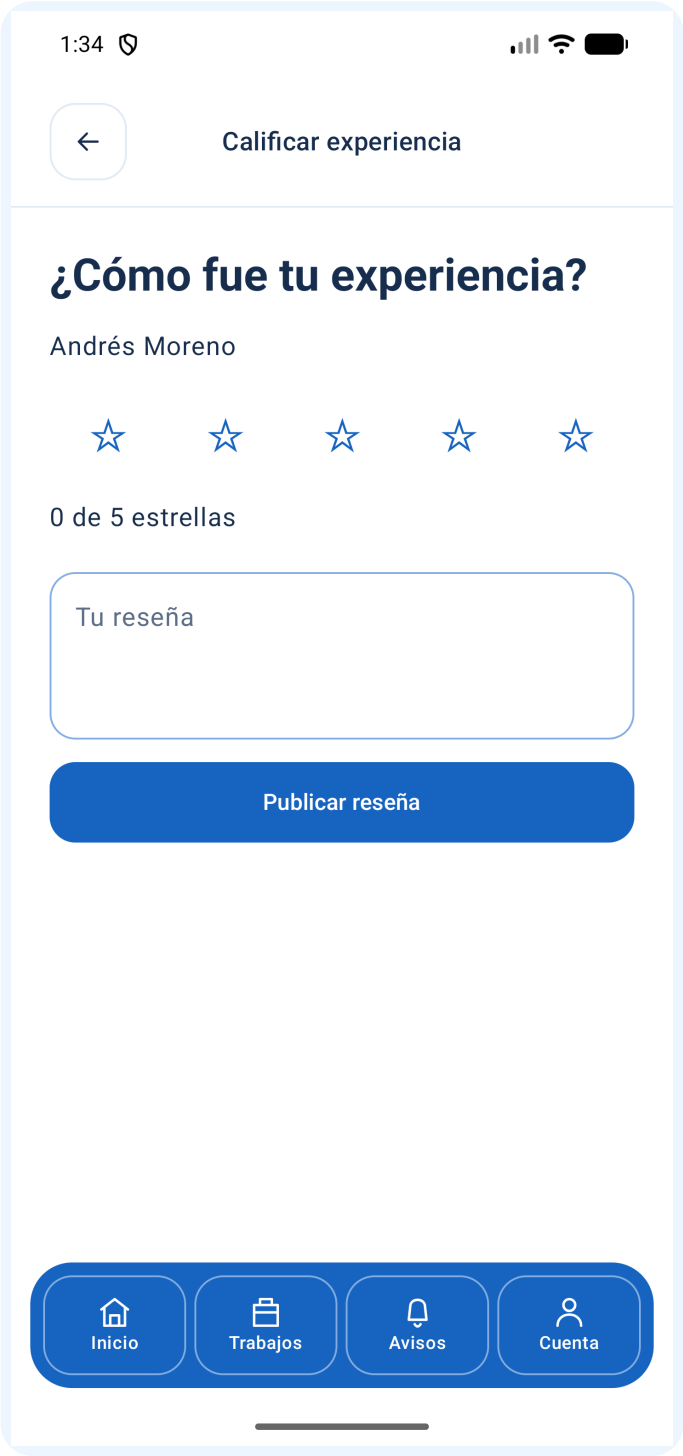
RESPECTO AL PROTOTIPO

La puntuación del perfil se calcula con las reseñas del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 30-profile.png

RF-32 · Calificaciones de trabajadores



IMPLEMENTACIÓN

Se eligen de 1 a 5 estrellas y un comentario obligatorio. Se permite una calificación por trabajo finalizado con trabajador asignado; el perfil calcula el promedio.

RESPECTO AL PROTOTIPO

La puntuación del perfil se calcula con las reseñas del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 25-review.png

RF-33 · Notificación por comentario recibido



IMPLEMENTACIÓN

Agregar un comentario a una publicación ajena crea un aviso dirigido a su autor. Los avisos del usuario activo se consultan en Avisos.

RESPECTO AL PROTOTIPO

Los avisos se almacenan en memoria, sin Firebase Cloud Messaging ni base de datos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 33-notifications.png

RF-34 · Notificación del resultado de moderación



IMPLEMENTACIÓN

El autor consulta los avisos generados después de verificar o rechazar solicitudes durante la evaluación.

RESPECTO AL PROTOTIPO

La captura corresponde a acciones reales de la secuencia de prueba, no a un listado aislado del prototipo.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 46-notification-moderation.png

RF-35 · Notificación por logro obtenido



IMPLEMENTACIÓN

La primera publicación del usuario de evaluación desbloquea una insignia y genera el aviso Nuevo logro. La captura muestra el aviso producido durante la prueba.

RESPECTO AL PROTOTIPO

Las insignias precargadas son coherentes con la actividad inicial. Los nuevos logros se notifican en memoria una sola vez.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 55-achievement-notice.png

RF-36 · Consulta de notificaciones



IMPLEMENTACIÓN

La lista muestra avisos propios y permite marcar uno o todos como leídos, revertir la lectura y abrir la publicación relacionada.

RESPECTO AL PROTOTIPO

Los avisos se almacenan en memoria, sin Firebase Cloud Messaging ni base de datos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 33-notifications.png

RF-37 · Estado de lectura de notificaciones



IMPLEMENTACIÓN

Cada aviso puede marcarse leído o no leído; también se pueden marcar todos. La captura muestra el estado y el control para revertirlo.

RESPECTO AL PROTOTIPO

El estado se conserva dentro del proceso, no entre reinicios, según la exención de almacenamiento.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 56-read-notice.png

RF-38 · Navegación desde una notificación



IMPLEMENTACIÓN

Al pulsar Ver detalle en un aviso asociado a una publicación se abre ese trabajo y el aviso se marca leído.

RESPECTO AL PROTOTIPO

Si la publicación fue eliminada se informa al usuario; los avisos de logros abren Mis logros.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 57-notice-detail.png

RF-39 · Actualización de notificaciones al consultar



IMPLEMENTACIÓN

Avisos observa el repositorio compartido mediante StateFlow y muestra los eventos existentes para la cuenta actual al entrar en la sección.

RESPECTO AL PROTOTIPO

Adaptación de fase: consulta del estado en memoria, sin base de datos ni Firebase Cloud Messaging.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 46-notification-moderation.png

03 · EVIDENCIAS Y TRAZABILIDAD

RF-40 · Dashboard de estadísticas personales



IMPLEMENTACIÓN

Muestra el total de publicaciones, progreso de finalización y tarjetas de búsqueda de trabajador, en progreso, finalizadas y en revisión. Más abajo aparecen rechazadas, participación y reputación.

RESPECTO AL PROTOTIPO

Se sustituyeron cifras aisladas por tarjetas explicativas. Activas se desglosa en buscando trabajador y en progreso; los datos provienen del usuario actual.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 34-dashboard.png

RF-41 · Acumulación de puntos por participación



IMPLEMENTACIÓN

Crear una publicación suma 10 puntos; su verificación, 20; comentar, 2; recibir un voto, 1. Se conserva un registro de recompensas para evitar duplicados.

RESPECTO AL PROTOTIPO

Se corrigieron los datos iniciales: los puntos se calculan a partir de las acciones de ejemplo, en vez de asignar cifras arbitrarias.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 35-levels.png

RF-42 · Niveles de usuario



IMPLEMENTACIÓN

Cuatro niveles: Vecino de 0 a 99; Colaborador de 100 a 499; Experto de 500 a 1499; Referente desde 1500. Mis logros muestra progreso y puntos para el siguiente nivel.

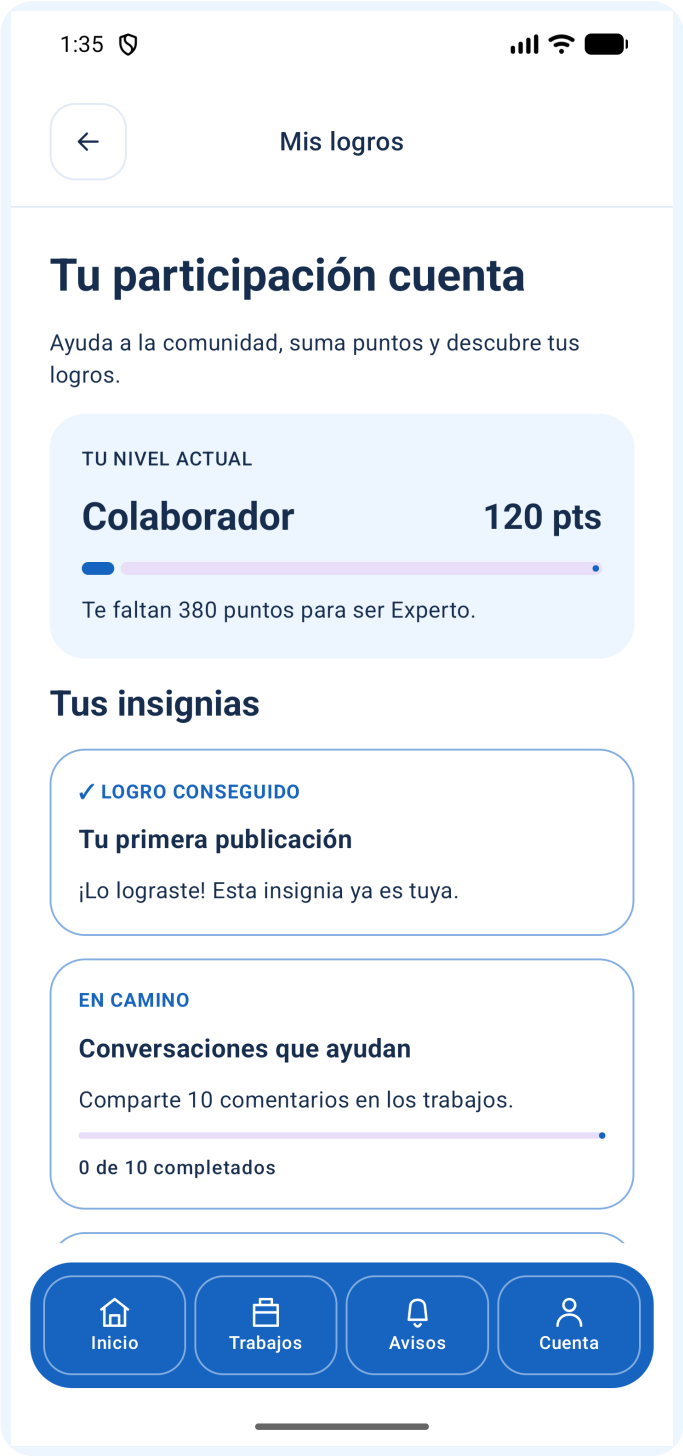
RESPECTO AL PROTOTIPO

Se evita otorgar puntos por auto-votos o por alternar repetidamente el mismo voto.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 35-levels.png

RF-43 · Obtención de insignias o badges



IMPLEMENTACIÓN

Se desbloquean primera publicación, diez comentarios y diez publicaciones verificadas; el desbloqueo genera una notificación.

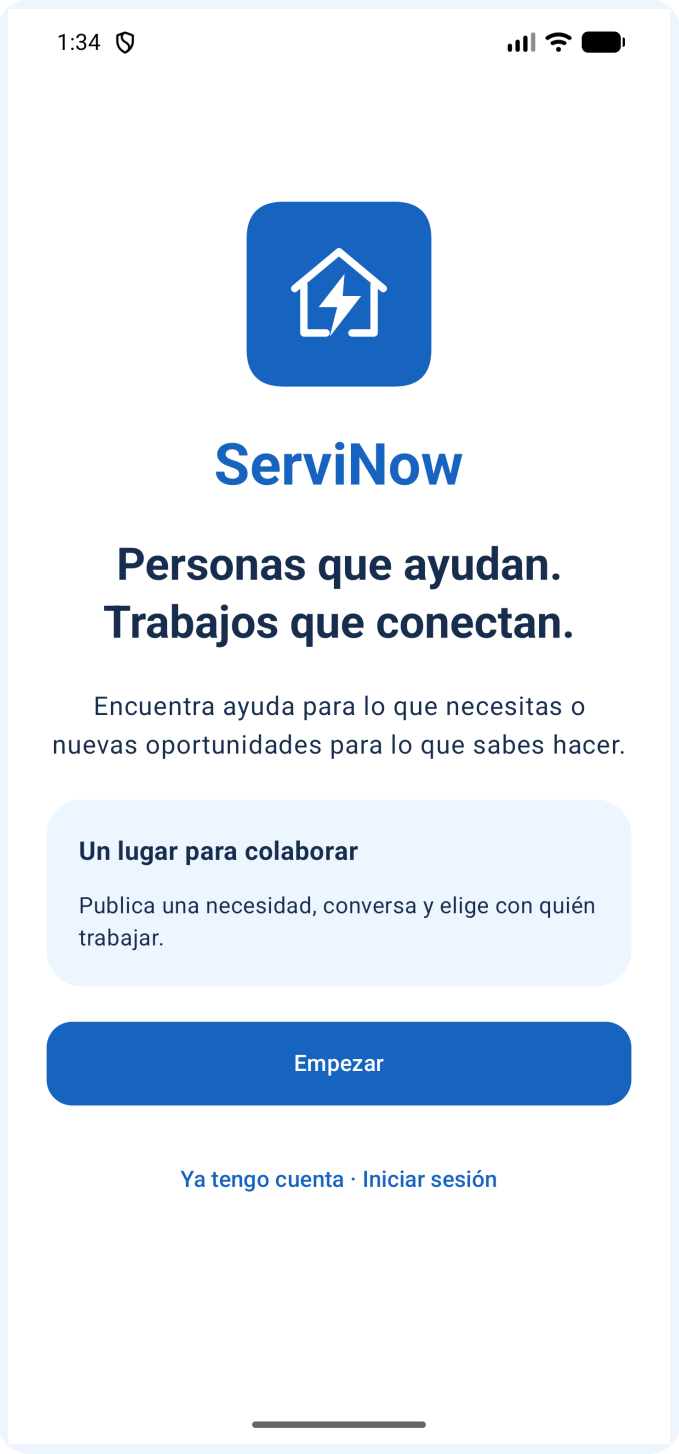
RESPECTO AL PROTOTIPO

Se eligieron logros verificables en la demostración. El destacado mensual era un ejemplo opcional del planteamiento inicial.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 35-achievements.png

RNF-01 · Plataforma y entorno de desarrollo



IMPLEMENTACIÓN

Aplicación Android ejecutada en Pixel 7 Pro virtual, con Android 17. El proyecto Gradle abre en Android Studio.

RESPECTO AL PROTOTIPO

La captura evidencia ejecución; los archivos Gradle y AndroidManifest complementan la verificación de plataforma.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 00-welcome.png

RNF-02 · Lenguaje y construcción de la interfaz



IMPLEMENTACIÓN

Interfaz construida con Kotlin y Jetpack Compose. MainActivity monta CommunityRoute; el código fuente es la evidencia técnica adicional.

RESPECTO AL PROTOTIPO

No se usan páginas HTML como interfaz principal ni como capturas de implementación.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 08-home.png

RNF-03 · Persistencia de datos



IMPLEMENTACIÓN

EXENTO EN FASE 2. Las publicaciones y relaciones se gestionan con InMemoryCommunityRepository.

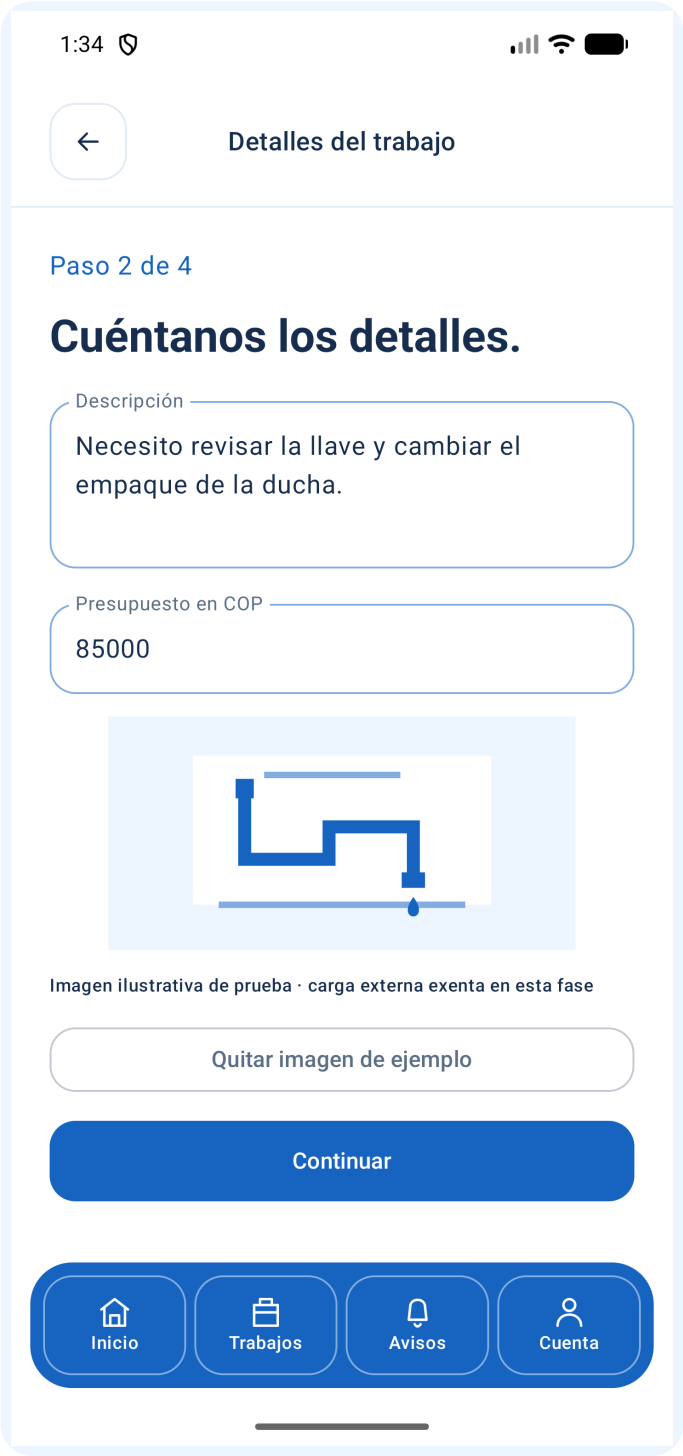
RESPECTO AL PROTOTIPO

No hay persistencia entre procesos en el recorrido de entrega. Esta captura muestra datos de la ejecución, no una base de datos.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 18-mine.png

RNF-04 · Servicio externo para imágenes



IMPLEMENTACIÓN

EXENTO EN FASE 2. Se utiliza una ilustración empaquetada con la app para representar la imagen del trabajo.

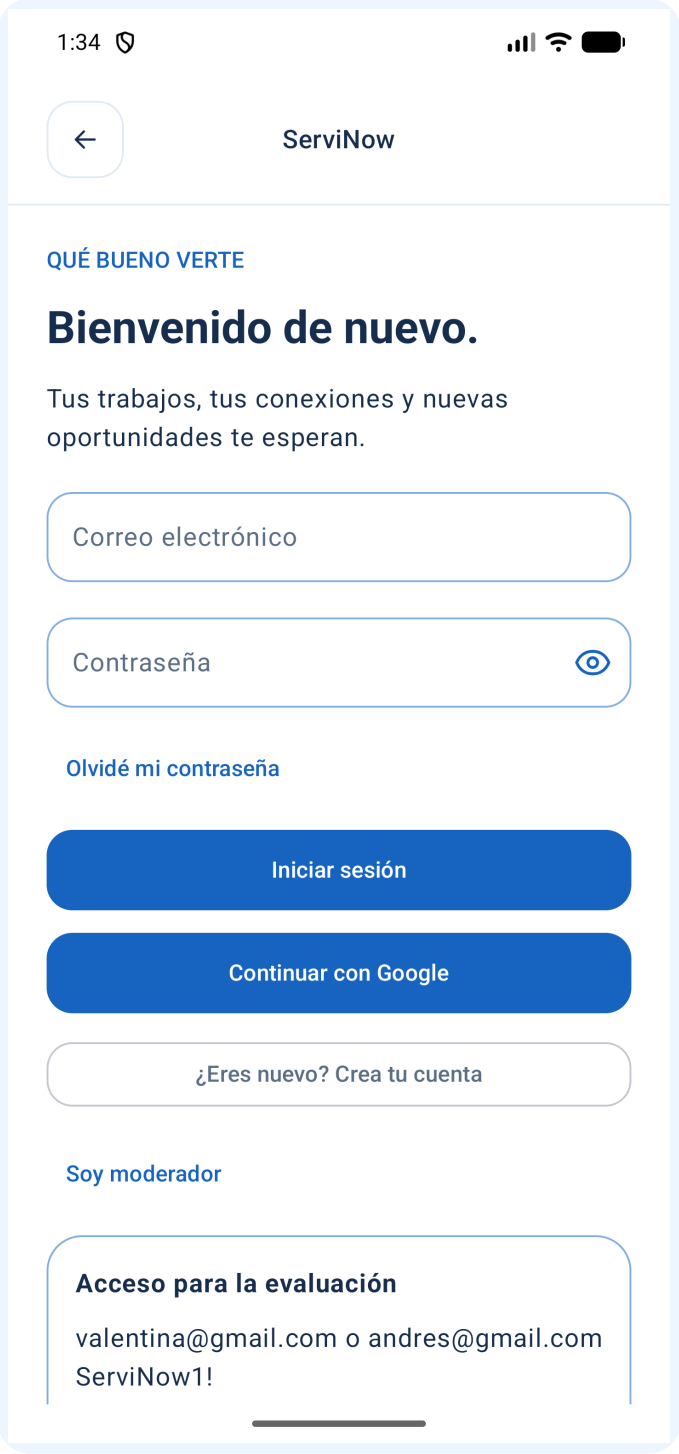
RESPECTO AL PROTOTIPO

No se integra Cloudinary, S3 ni carga externa. No se afirma que este requisito de producción esté implementado.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 14-create-details.png

RNF-05 · Servicios de autenticación y almacenamiento



IMPLEMENTACIÓN

ADAPTACIÓN DE FASE. Registro, acceso y recuperación operan con usuarios y claves de demostración en memoria.

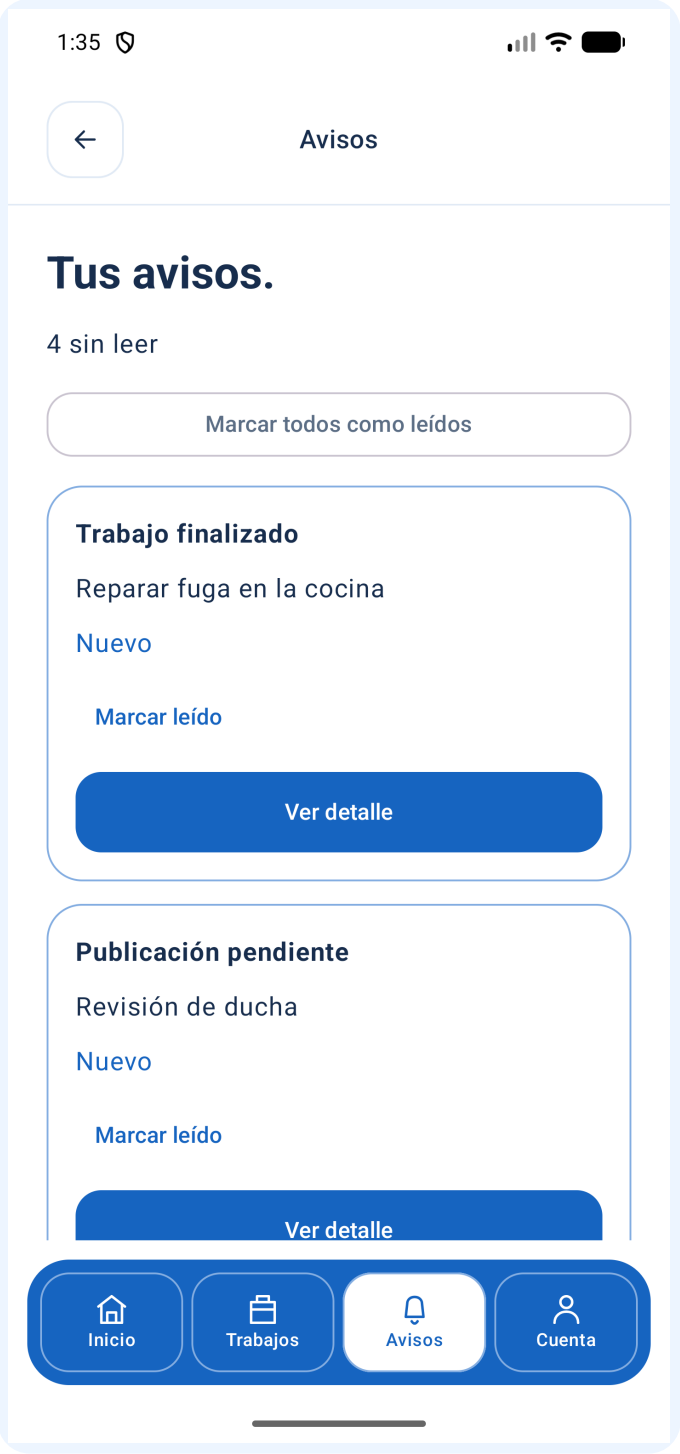
RESPECTO AL PROTOTIPO

No hay OAuth de Google, backend ni correo real. No se presenta como autenticación de producción.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 01-login.png

RNF-06 · Consulta de notificaciones sin dependencia de push



IMPLEMENTACIÓN

Los avisos se consultan desde el estado compartido. No se requiere ni utiliza push para el flujo mínimo.

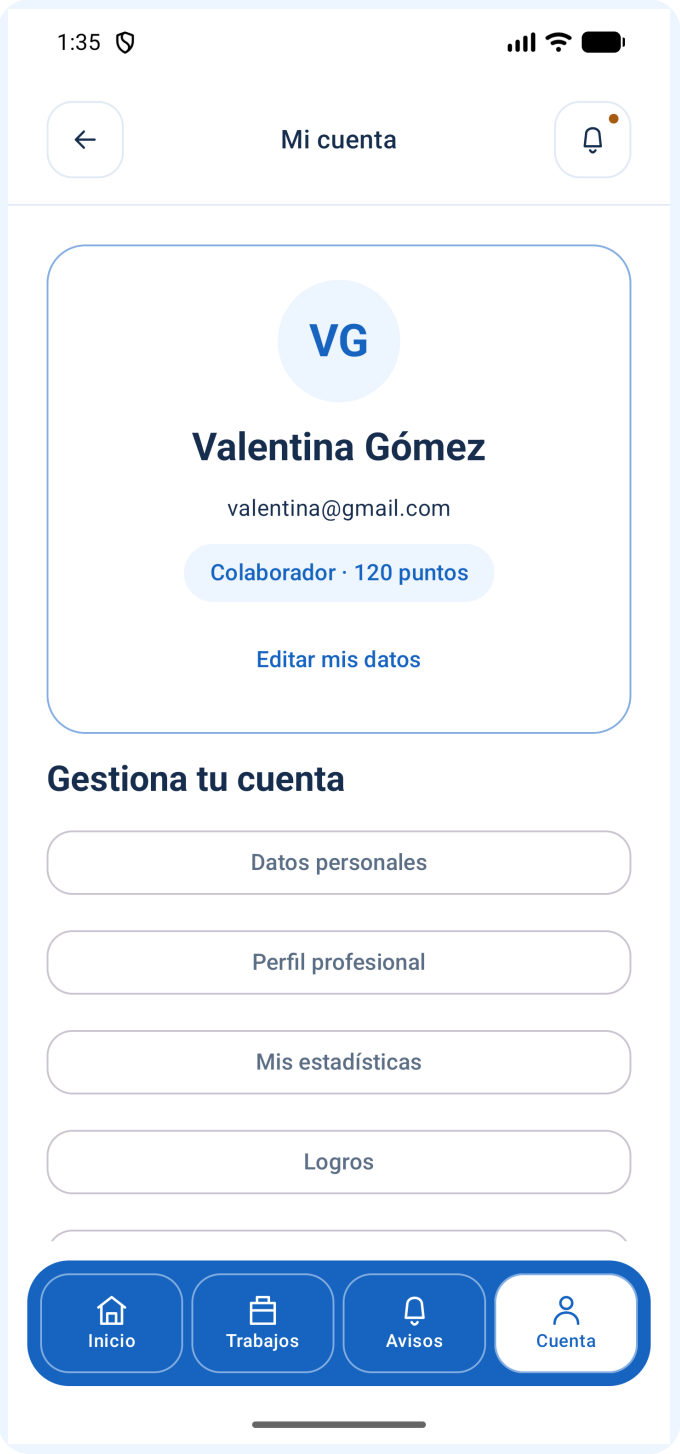
RESPECTO AL PROTOTIPO

La consulta a base de datos se sustituye por memoria, conforme a la consigna actual.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 33-notifications.png

RNF-07 · Interfaz amigable y elegante



IMPLEMENTACIÓN

Interfaz blanca y azul, tarjetas únicas con bordes redondeados, scroll, jerarquía tipográfica y controles con texto.

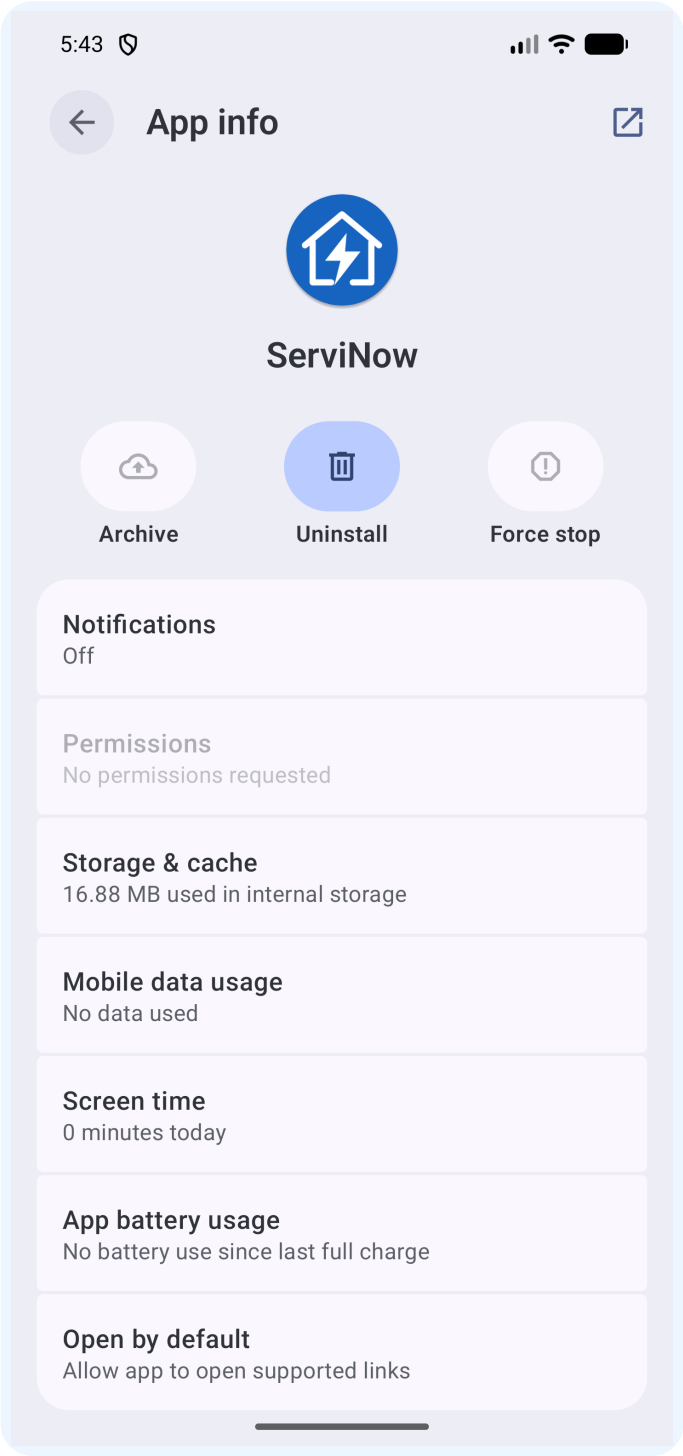
RESPECTO AL PROTOTIPO

Se revisaron capturas del emulador. No se afirma haber realizado un estudio de usabilidad ni una certificación de accesibilidad.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 36-account.png

EV-01 · Identidad: ícono de la app



IMPLEMENTACIÓN

La información de la app instalada en Android muestra el ícono propio de casa y rayo, junto al nombre ServiNow.

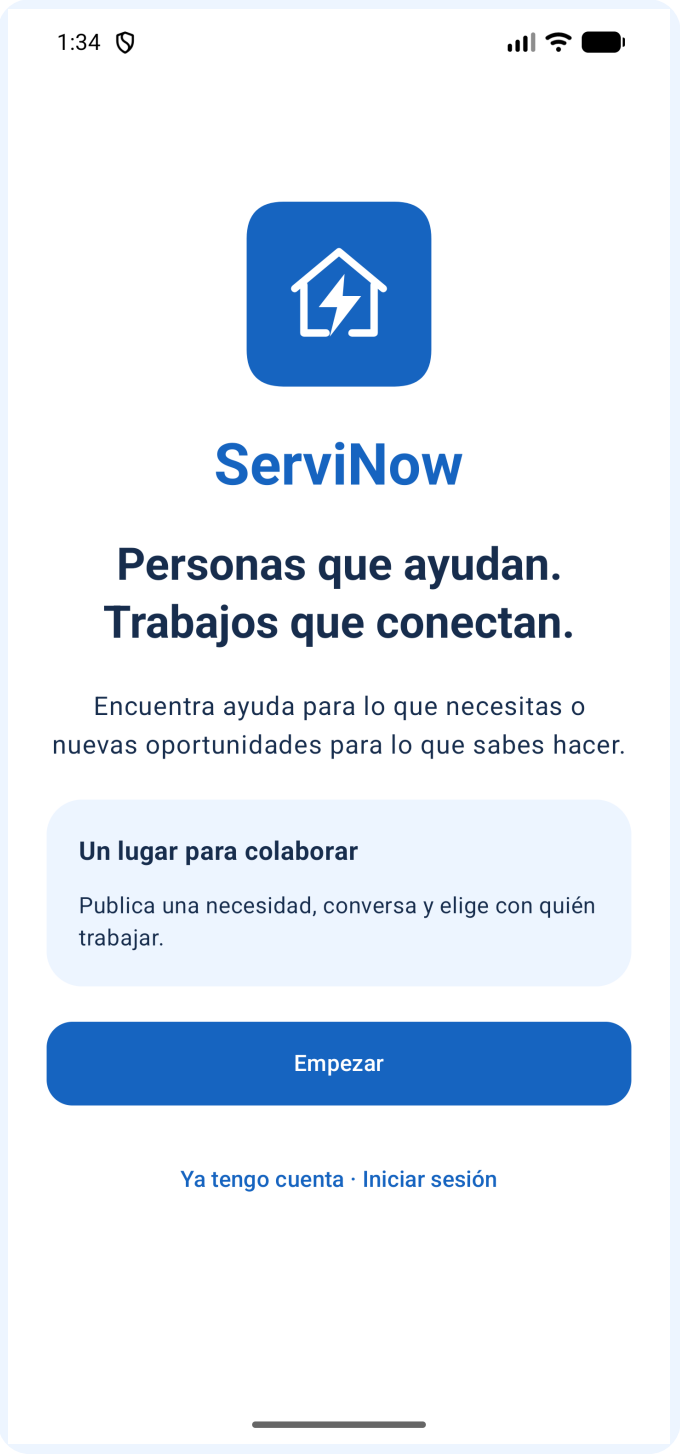
RESPECTO AL PROTOTIPO

Se reemplazó el ícono predeterminado con un vector adaptable en blanco y azul.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 00-icon.png

EV-02 · Pantalla inicial con logo



IMPLEMENTACIÓN

Presenta el logo, propósito de la app y accesos a la guía o al inicio de sesión.

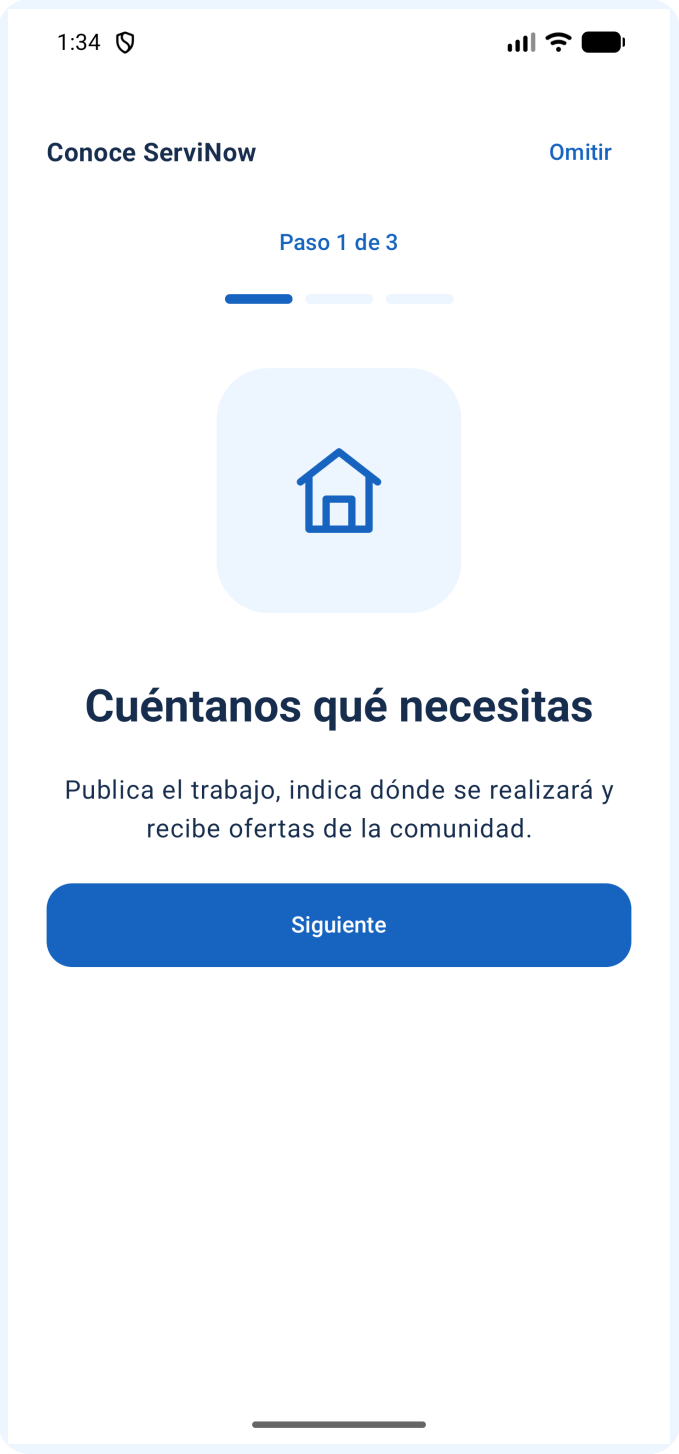
RESPECTO AL PROTOTIPO

Se añadió tras la revisión para dar una entrada reconocible al producto.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 00-welcome.png

EV-03 · Bienvenida: publicar



IMPLEMENTACIÓN

Primer paso de una guía de tres pantallas. Permite continuar u omitir.

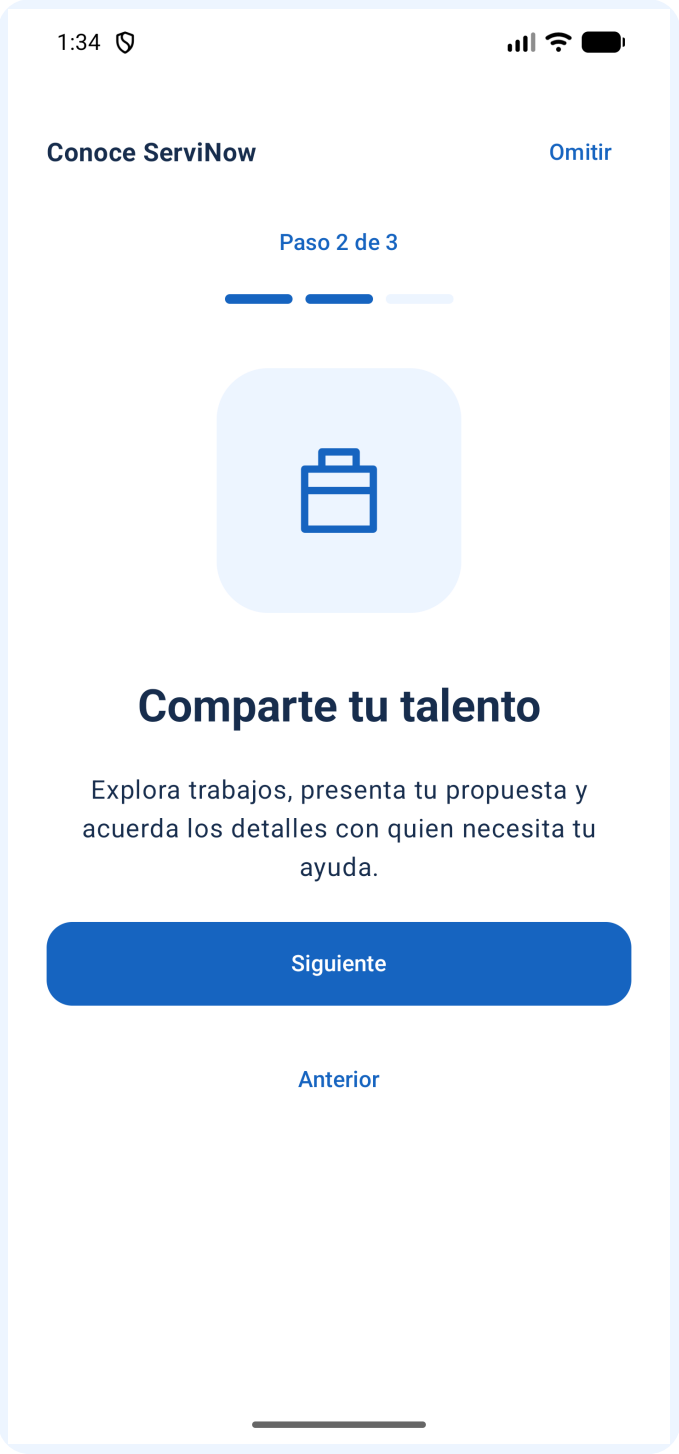
RESPECTO AL PROTOTIPO

La guía explica la propuesta antes del registro.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 00-onboarding-1.png

EV-04 · Bienvenida: trabajar



IMPLEMENTACIÓN

Segundo paso: explica cómo buscar oportunidades y enviar propuestas.

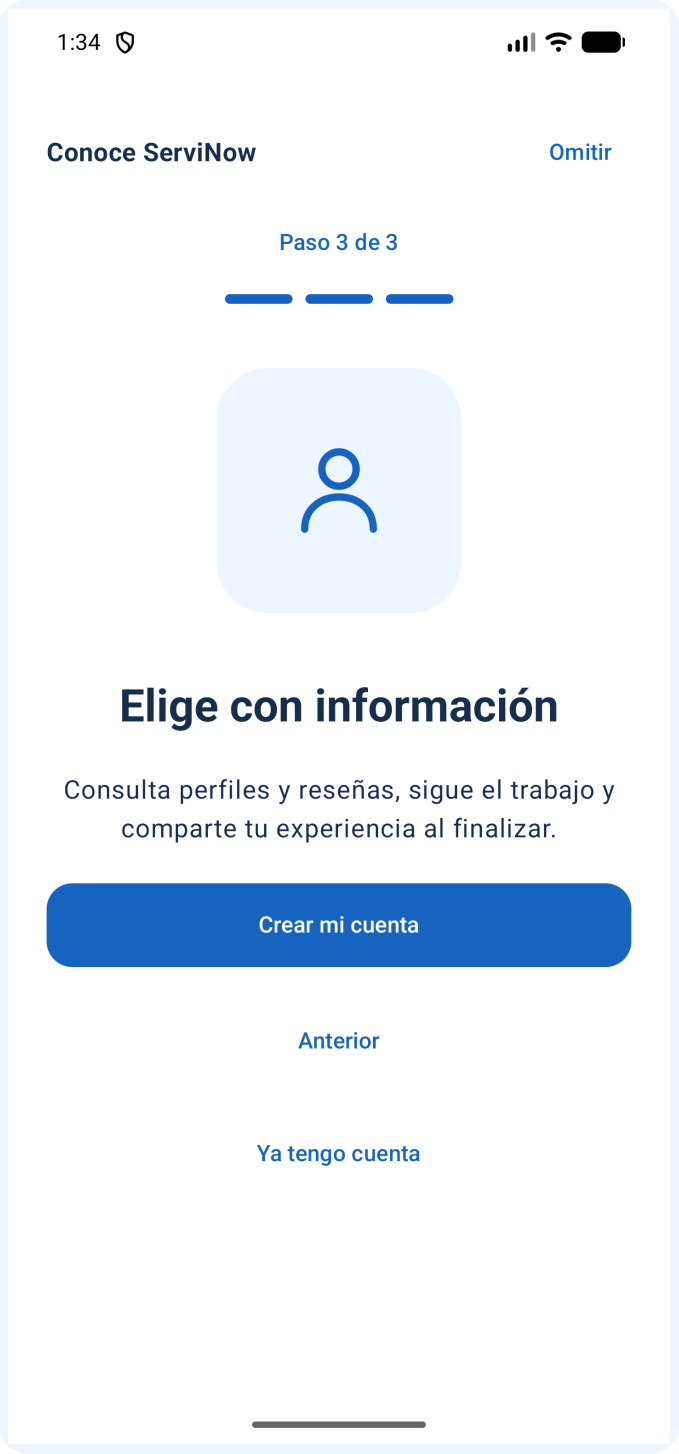
RESPECTO AL PROTOTIPO

Permite volver al paso anterior y conserva el paso al recrear la Activity.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 00-onboarding-2.png

EV-05 · Bienvenida: elegir



IMPLEMENTACIÓN

Tercer paso: presenta perfiles, reseñas y seguimiento. Ofrece crear cuenta o entrar.

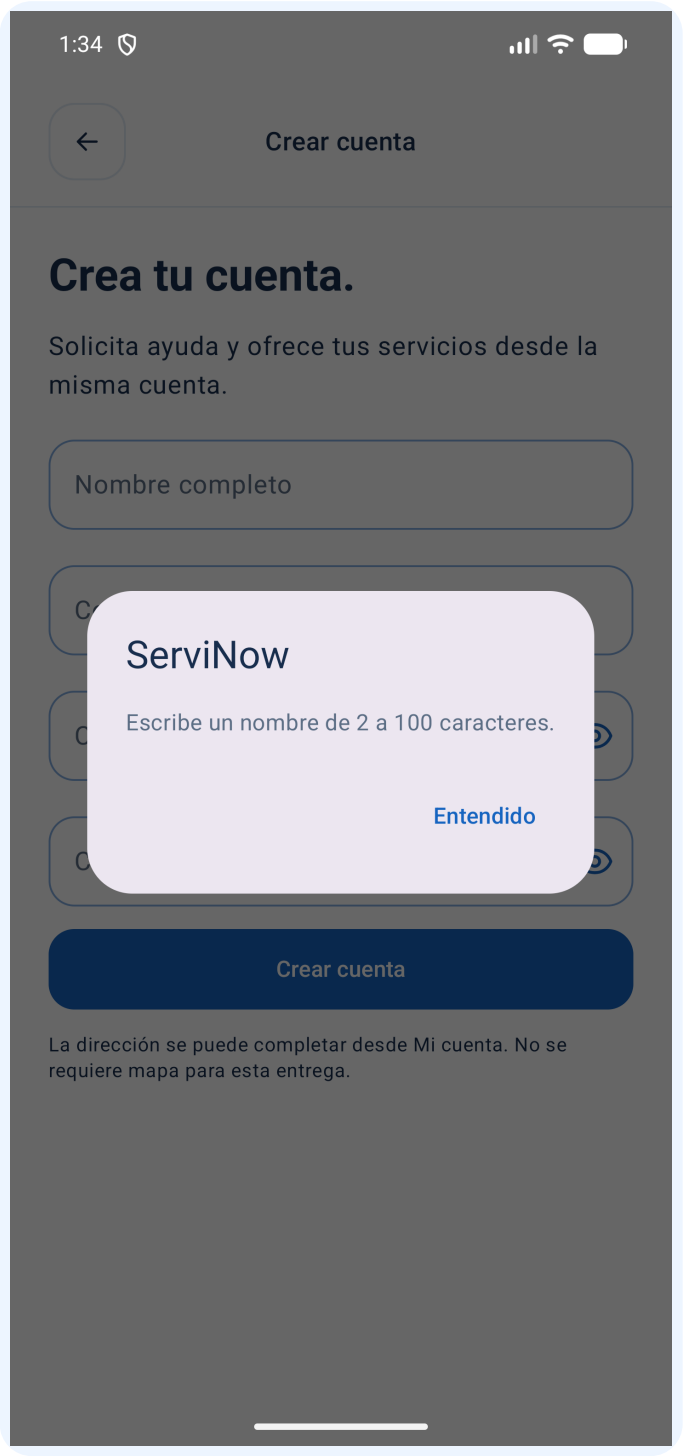
RESPECTO AL PROTOTIPO

La bienvenida se repite al iniciar un nuevo proceso; no se guarda una preferencia persistente en esta fase.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 00-onboarding-3.png

EV-06 · Validación de campos obligatorios



IMPLEMENTACIÓN

Enviar el registro vacío muestra un diálogo que solicita un nombre válido.

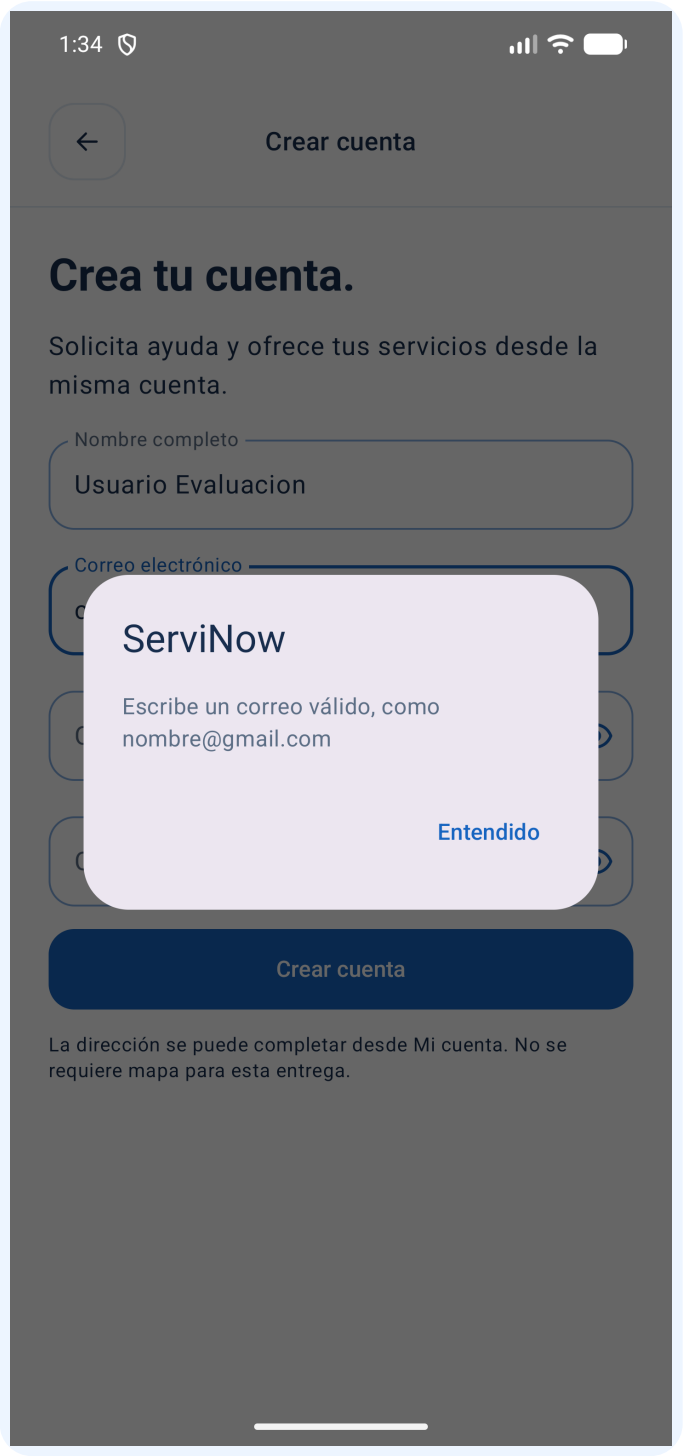
RESPECTO AL PROTOTIPO

La retroalimentación aparece al enviar el formulario, según lo solicitado.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 50-empty-form.png

EV-07 · Validación de correo



IMPLEMENTACIÓN

El formulario muestra el error de formato del correo.

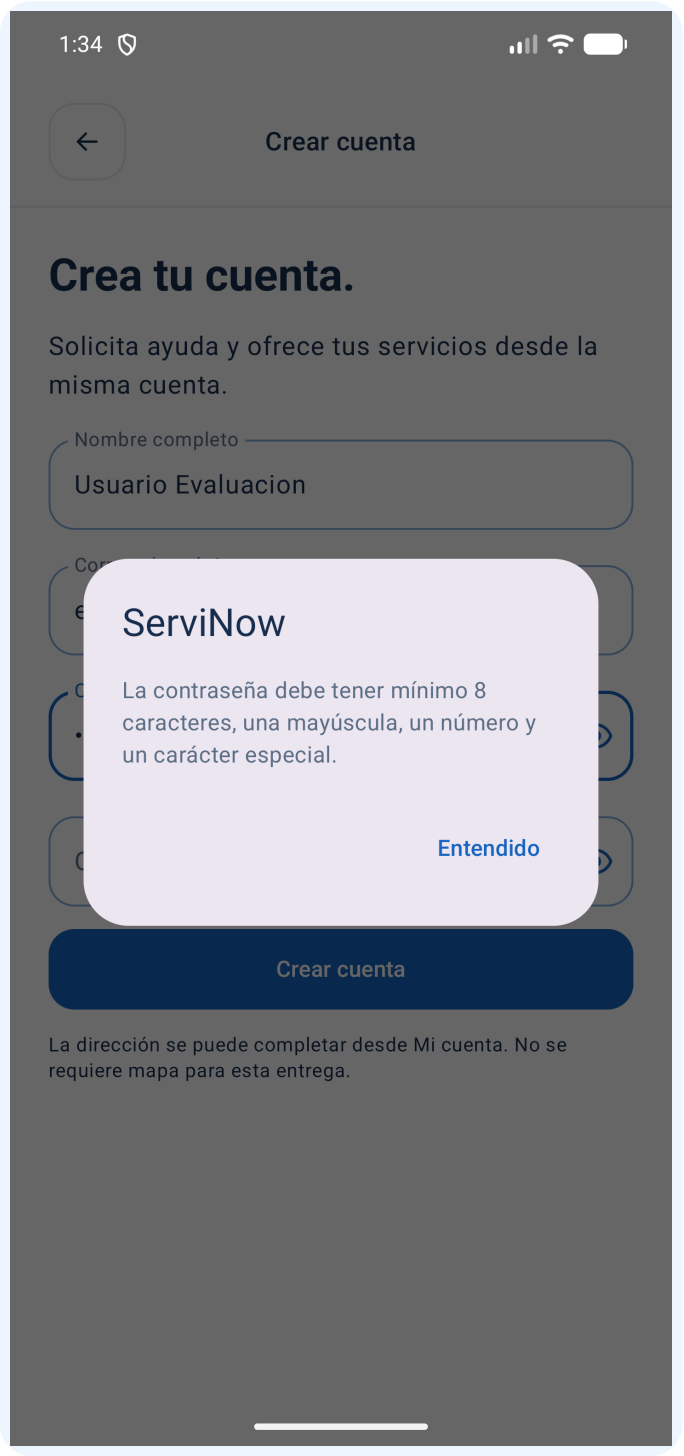
RESPECTO AL PROTOTIPO

Solo se admiten gmail.com, hotmail.com, outlook.com y yahoo.com; esto no verifica propiedad ni existencia de la cuenta.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 51-invalid-email.png

EV-08 · Reglas de contraseña



IMPLEMENTACIÓN

El registro exige al menos 8 caracteres, una mayúscula, un número y un símbolo.

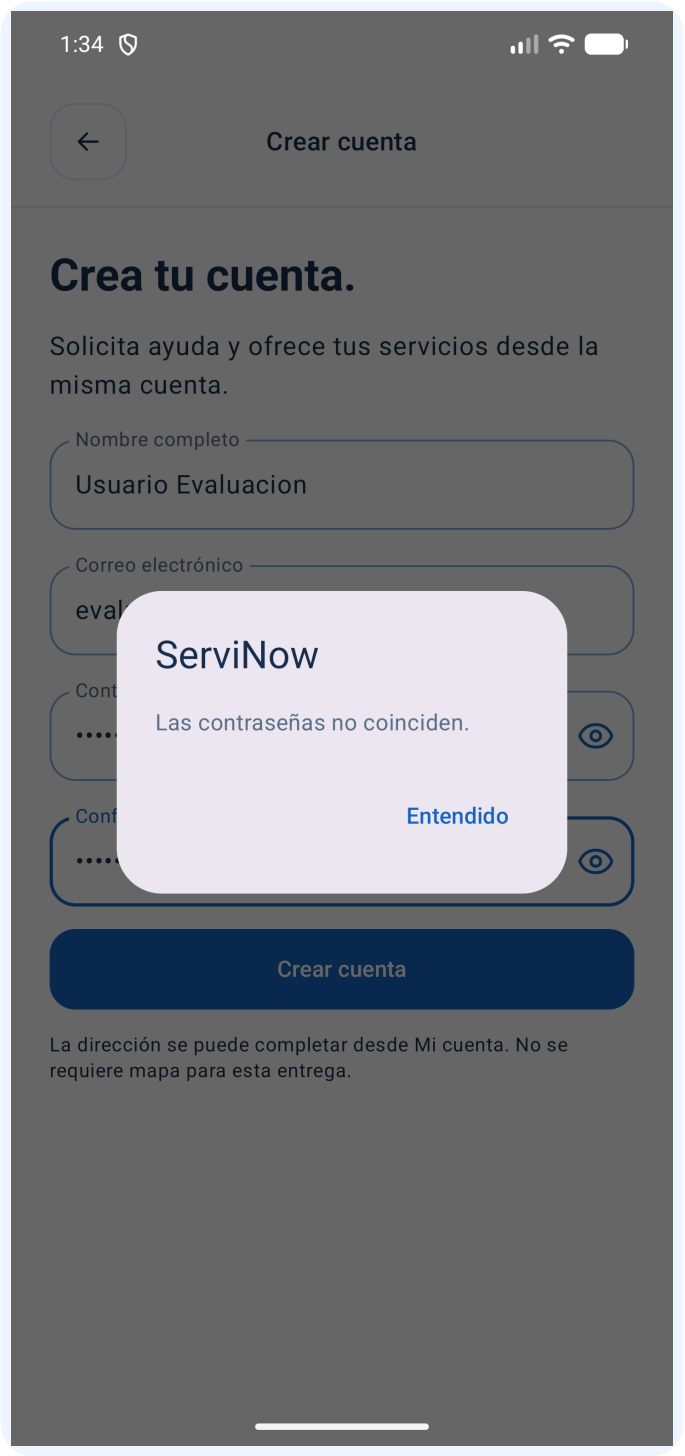
RESPECTO AL PROTOTIPO

El mensaje es emergente; no se agrega como bloque permanente de instrucciones.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 52-password-rules.png

EV-09 · Confirmación de contraseña



IMPLEMENTACIÓN

El registro rechaza contraseñas distintas con un mensaje explícito.

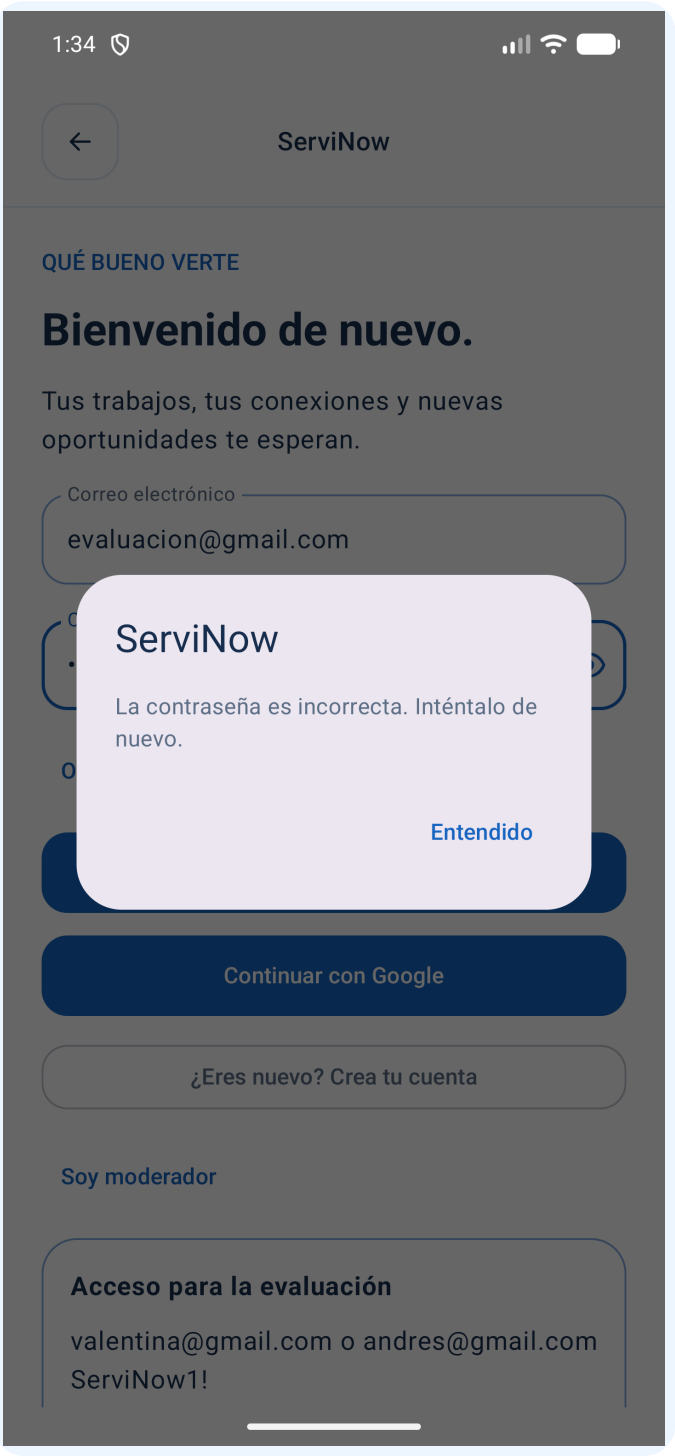
RESPECTO AL PROTOTIPO

Se evita crear la cuenta cuando la confirmación no coincide.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 53-password-mismatch.png

EV-10 · Credenciales incorrectas



IMPLEMENTACIÓN

Una contraseña incorrecta impide el acceso y produce un diálogo de error.

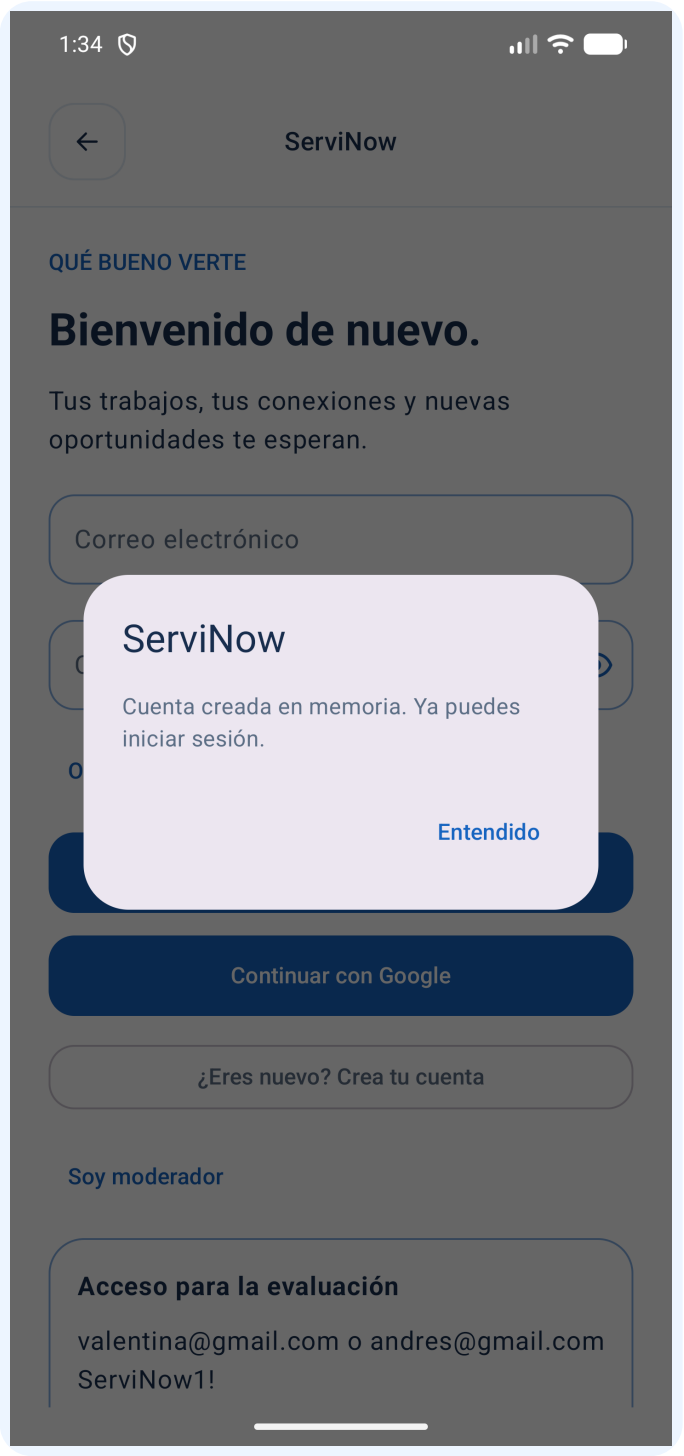
RESPECTO AL PROTOTIPO

Se diferencian las reglas del registro y la comprobación de credenciales al entrar.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 54-login-error.png

EV-11 · Registro confirmado



IMPLEMENTACIÓN

Tras completar datos válidos se crea el usuario y se confirma el resultado.

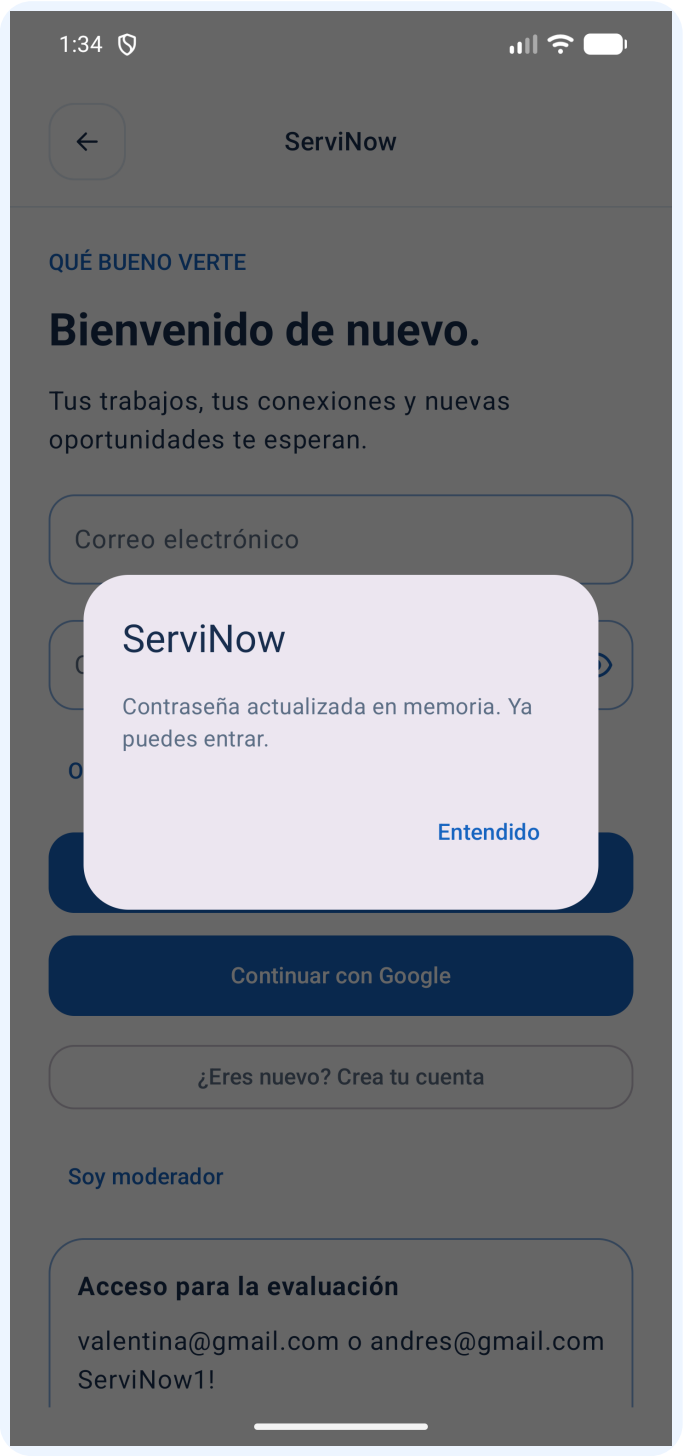
RESPECTO AL PROTOTIPO

La prueba continúa entrando con la cuenta creada.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 03-registration-success.png

EV-12 · Contraseña actualizada



IMPLEMENTACIÓN

Se confirma el cambio mediante el enlace simulado y se entra con la nueva clave.

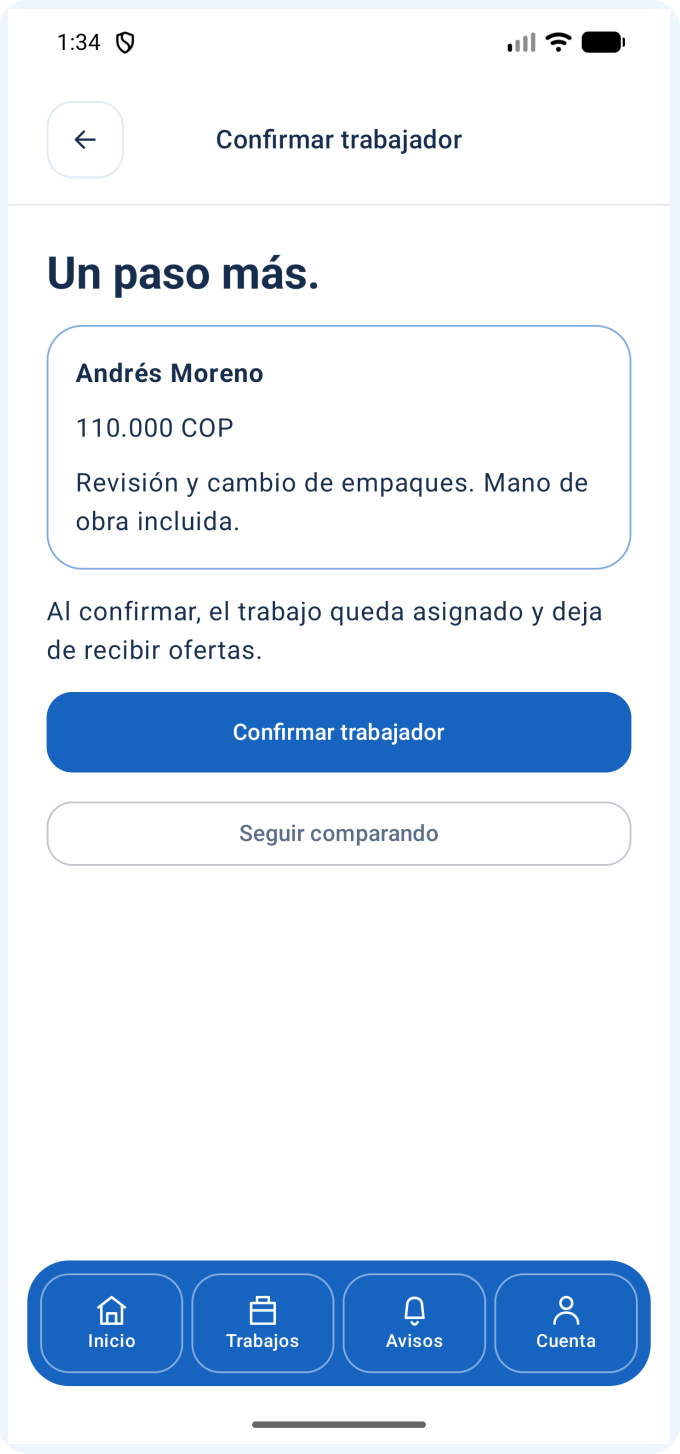
RESPECTO AL PROTOTIPO

La captura no implica envío de correo real.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 07-password-saved.png

EV-13 · Aceptar una oferta



IMPLEMENTACIÓN

El autor confirma al trabajador; el trabajo queda en progreso y se genera un aviso.

RESPECTO AL PROTOTIPO

Se conserva una asignación exclusiva por publicación.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 22-confirm.png

EV-14 · Seguimiento del trabajo



IMPLEMENTACIÓN

Presenta el estado y la persona asignada; permite consultar perfil y conversación.

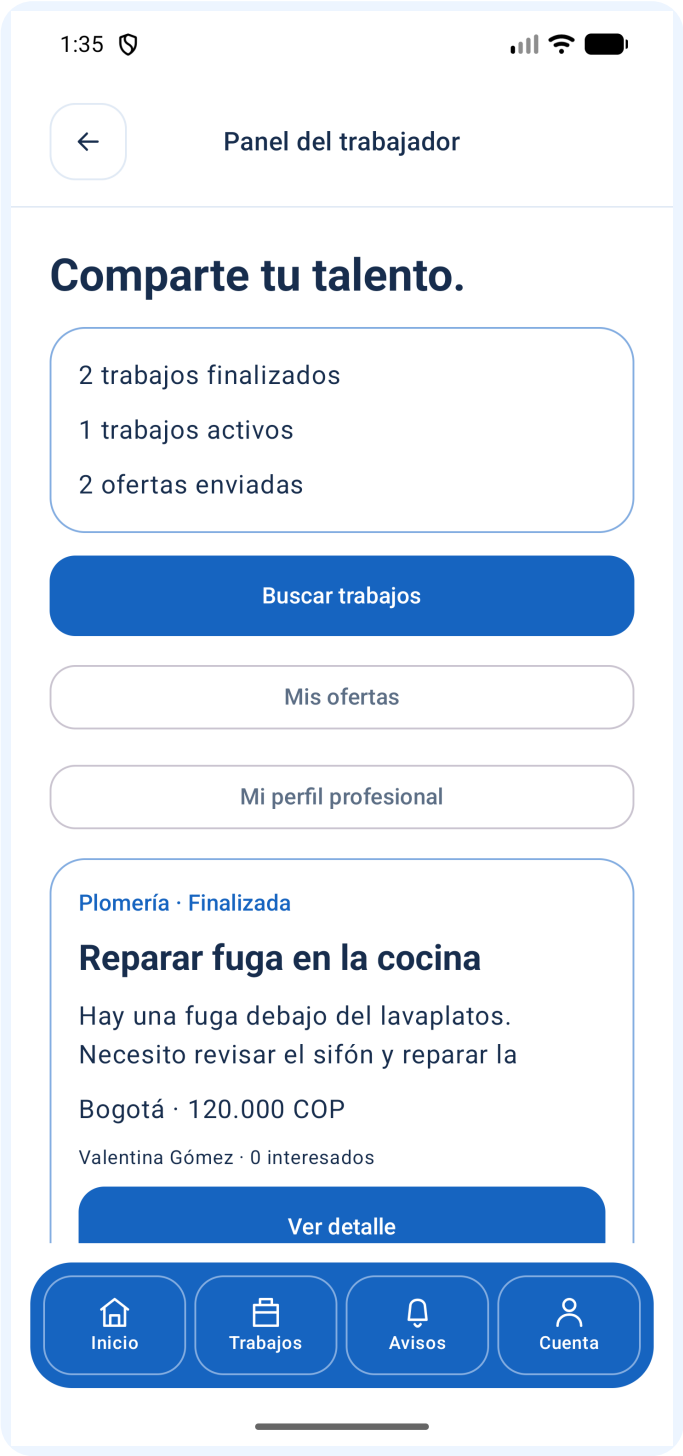
RESPECTO AL PROTOTIPO

El seguimiento usa los estados del repositorio, no geolocalización en tiempo real.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 23-tracking.png

EV-15 · Panel del trabajador



IMPLEMENTACIÓN

El usuario consulta sus trabajos y accede a oportunidades.

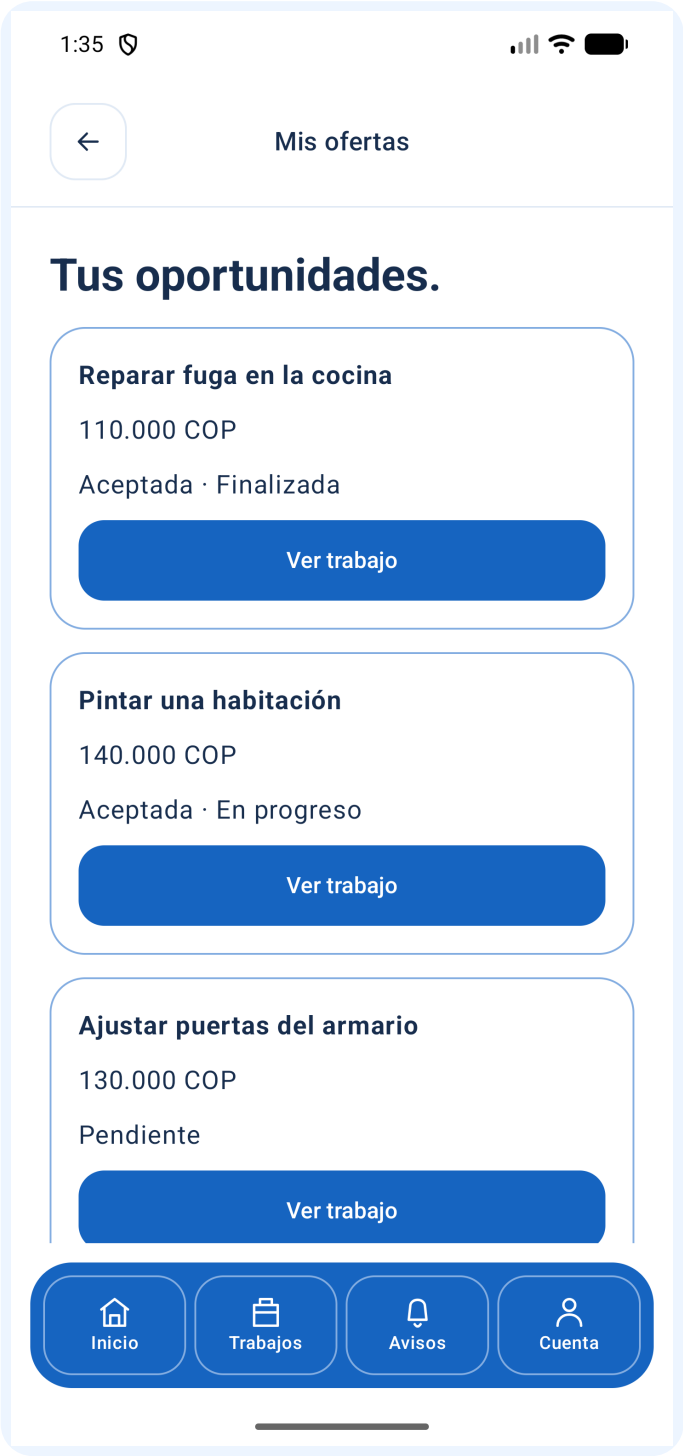
RESPECTO AL PROTOTIPO

Solicitante y trabajador son formas de participación del mismo rol Usuario.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 27-worker.png

EV-16 · Mis ofertas



IMPLEMENTACIÓN

Muestra propuestas enviadas y su estado.

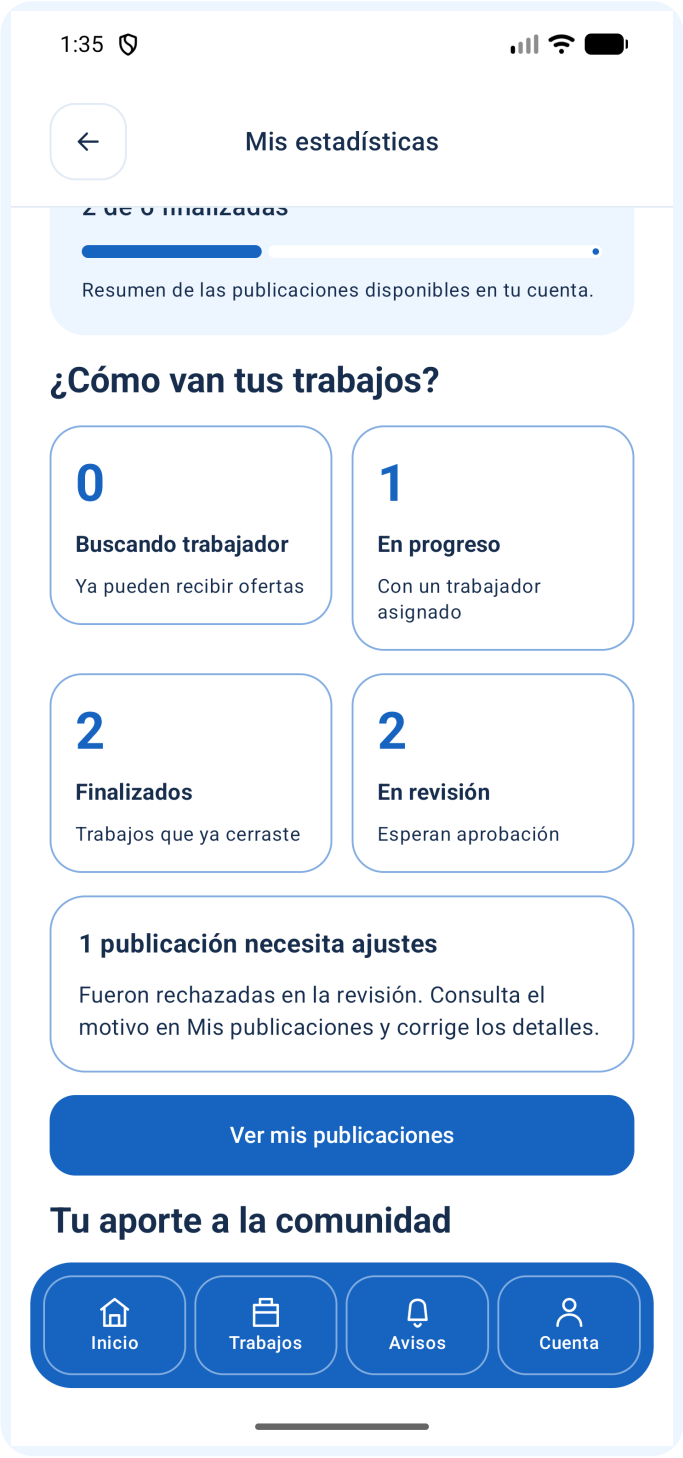
RESPECTO AL PROTOTIPO

Los datos están relacionados con las publicaciones del repositorio.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 29-my-offers.png

EV-17 · Estadísticas: participación



IMPLEMENTACIÓN

Complementa los estados de trabajos con comentarios, interés recibido y reputación.

RESPECTO AL PROTOTIPO

Se aclaró el significado de cada cifra y se añadió acceso a los logros.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 34-dashboard-bottom.png

EV-18 · Búsqueda sin resultados



IMPLEMENTACIÓN

Una búsqueda sin coincidencias muestra un mensaje y permite restablecer filtros.

RESPECTO AL PROTOTIPO

La captura se realiza sin teclado para mostrar la retroalimentación completa.

EVIDENCIA

Captura del emulador Android ejecutando la aplicación Kotlin/Compose. Archivo: 44-empty.png

04 · REQUISITOS TÉCNICOS

Arquitectura y condiciones de fase

Kotlin, Compose y MVVM

MainActivity aloja CommunityRoute. Navigation Compose gestiona la pila con NavHost y NavController. CommunityViewModel emite eventos de navegación y realiza las acciones; ya no mantiene un historial manual. Compose obtiene el ViewModel con hiltViewModel(), sin ViewModelProvider en la Activity. CommunityRepository define el contrato en dominio; InMemoryCommunityRepository lo implementa en datos. Hilt vincula la interfaz y entrega una instancia compartida. StateFlow actualiza las pantallas.

Reglas y consistencia

Las restricciones de rol, propiedad, estados, duplicados y validación se verifican al operar los datos. Las ofertas, publicaciones, comentarios, avisos y reseñas usan identificadores; no son listas independientes sin relación.

Persistencia, imágenes, ubicación y permisos

Exentos por la consigna: base de datos, almacenamiento externo, carga de imágenes, permisos, mapas y sesiones. Se incluyen imagen ilustrativa, coordenadas de referencia y cuentas de prueba en memoria. El mapa opcional no condiciona el recorrido principal.

GitHub y contribuciones

El enlace del repositorio figura en la portada. Cada integrante debe realizar y conservar sus contribuciones reales con su identidad; la asistencia de IA no sustituye la participación del grupo. No se fabrican commits a nombre de los integrantes.

Legibilidad y presentación

Interfaz en español, fondo blanco, acentos y bordes azules, botones redondeados, una tarjeta por bloque, icono de ojo para contraseñas y mensajes emergentes. Se conserva scroll para pantallas pequeñas y teclado.

05 · DIFERENCIAS

Cambios frente a la primera fase

Identidad visual

Se incorporaron un logo vectorial propio, ícono adaptable, pantalla inicial y onboarding de tres pasos. Se pasó de las primeras propuestas oscuras, naranja o verde a la variante blanca y azul elegida por el usuario. Se redondearon controles y la barra inferior para separar opciones con claridad.

Tarjetas y encabezados

Se eliminaron recuadros duplicados en perfiles y reseñas. La campana se reservó para perfil/cuenta; las pantallas de recuperación no muestran avisos ajenos al flujo.

Datos conectados

El prototipo mostraba ejemplos estáticos. La entrega usa un repositorio en memoria: comentarios generan avisos, ofertas asignan trabajadores, moderación cambia la visibilidad y reseñas actualizan perfiles.

Formularios y confirmaciones

Se añadieron validaciones de contraseña, correo, campos obligatorios y presupuesto. Eliminar publicaciones/cuentas, aceptar ofertas y finalizar trabajos requieren acciones explícitas.

Simplificaciones de alcance

Registro sin mapa obligatorio; ciudad de referencia para solicitudes; imagen de ejemplo en lugar de subida externa; currículum estructurado en lugar de archivo obligatorio; recuperación con enlace de prueba. Estas diferencias responden a las exenciones de la consigna.

Servicios no afirmados como completos

No se afirma que existan autenticación de Google, correo real, backend, permisos o persistencia de sesiones. El estado de error de conexión es una pantalla de referencia, no una prueba de un servicio remoto.

06 · USO DE IA

Herramientas y propósito

Herramienta utilizada

Codex, asistente de programación de OpenAI, utilizado en esta conversación como agente principal. No se atribuye trabajo a otros agentes ni a herramientas de IA que no constan en el historial.

Apoyo en el diseño

Interpretación de requisitos y referencias visuales; adaptación del diseño blanco y azul; separación de tarjetas, navegación y mensajes de validación.

Apoyo en implementación

Generación y ajuste de Kotlin/Compose, ViewModels, modelos, repositorios e inyección Hilt/KSP. Implementación de flujos en memoria para la fase académica y restricciones de roles/propiedad.

Apoyo en verificación

Creación de pruebas unitarias y pruebas instrumentadas de flujos, Navigation Compose y onboarding. Lectura de errores de compilación, corrección de incompatibilidades y generación de capturas reales desde Android. Formateo inicial con ktfmt; Gradle y Android Emulator son herramientas de desarrollo, no agentes de IA.

Apoyo documental

Organización de requisitos y evidencias, redacción del registro de cambios y del uso de IA. Generación del PDF con ReportLab y revisión visual de páginas renderizadas.

Límite de responsabilidad

El asistente apoya, pero el grupo debe comprender, revisar y ajustar el código. Este documento no certifica que los tres integrantes hayan hecho una revisión manual completa; esa validación humana corresponde al grupo antes de entregar.

06 · USO DE IA

Prompts relevantes del trabajo

Diseño y requisitos

“Debes hacer los mockups cumpliendo con todos los requisitos [...] que se vean más amigables [...] y elegante”. Propósito: derivar pantallas y jerarquía visual de la especificación.

Interacción y validaciones

“La contraseña debe tener [...] mínimo [...] mayúscula y número”; “el mensaje solo debe salir [...] cuando se registre”; “esto tiene que ser un ojo [...] ver contraseña”. Propósito: precisar validación y controles del formulario.

Navegación funcional

“Se ve estático, debes hacer que sea funcional la app, no solo [...] vistas”. Propósito: conectar pantallas, acciones y estados.

Navigation Compose y bienvenida

“Aplica el navigation compose” y “realiza las 2 primeras”, referido al logo y onboarding. Propósito: sustituir el historial manual por Navigation Compose e incorporar identidad y guía inicial.

Arquitectura de clase

“Revisa esto y aprende”, junto con la guía de ViewModels/repositorios, y “organiza el proyecto como debe quedar”. Propósito: separar presentación, dominio, datos e inyección.

Alcance de esta entrega

Se proporcionó la consigna de fase dos y se confirmó: “Sí, completa la app y el PDF”. Propósito: completar flujos con datos locales y producir evidencias por requisito.

Trazabilidad

Los fragmentos se normalizan en ortografía y se acortan para facilitar la lectura; no representan un registro exhaustivo de todos los mensajes. Los resultados se validan contra el comportamiento de la app y las pruebas, no contra la mera aceptación de una respuesta de IA.

Dificultades y correcciones

Pantallas sin operaciones reales

La versión inicial validaba formularios, pero no creaba cuentas ni ejecutaba moderación. Se implementó un repositorio compartido en memoria y se conectaron registro, acceso, revisión, ofertas, comentarios y reseñas.

Mapa en blanco

Se detectó un área de mapa sin contenido. Se ajustaron el tamaño del WebView, los recursos Leaflet incluidos y la actualización del tamaño. Se verificó en el emulador que tocar otro lugar cambiaba las coordenadas.

Errores durante la reorganización

La extracción de clases produjo imports duplicados y desajustes en dobles de prueba. Se corrigieron imports, constructores e interfaces; se repitió la compilación y las pruebas.

Compatibilidad de pruebas Android

La infraestructura inicial de pruebas usaba versiones antiguas y falló en Android 17 al acceder a InputManager. Se actualizaron las bibliotecas de pruebas y se volvió a ejecutar el recorrido instrumentado.

Compilación de estados opcionales

Al conectar el flujo completo se detectaron accesos a publicaciones que podían ser nulas. Se añadieron comprobaciones explícitas antes de consultar sus datos.

Observaciones de la revisión

Se detectó navegación manual, ícono predeterminado y ausencia de onboarding. Se integró NavHost/NavController, hiltViewModel(), logo adaptable y guía de tres pasos. La prueba comprueba volver, pestañas, recreación y salida. El PDF anterior renumeraba requisitos: ahora conserva los IDs originales.

Aislamiento de pruebas

La ejecución conjunta falló porque la prueba de recuperación modificaba la clave usada por la siguiente prueba. Se cambió la cuenta de la prueba de navegación y se repitió el conjunto completo, con resultado satisfactorio.

Prevención de resultados engañosos

Se diferenciaron el enlace de recuperación simulado, las imágenes de ejemplo y los datos en memoria de servicios remotos reales. Las capturas se tomaron de Android, no del prototipo HTML.

07 · VERIFICACIÓN

Cómo se comprobó el resultado

Compilación

Se ejecutó Gradle para generar la aplicación debug y el paquete de pruebas Android. Resultado final: compilación satisfactoria.

Pruebas unitarias

43 pruebas aprobadas, 0 fallos. Incluyen validaciones previas y pruebas nuevas de registro, recuperación, rol, moderación, notificaciones, votos, ofertas, reseñas, eliminación y estado pendiente.

Prueba instrumentada

4 recorrido(s) instrumentado(s) aprobado(s), 0 fallos finales. Se operan los formularios de registro, login y reseña mediante controles Compose; el resto del recorrido se dirige mediante el mismo ViewModel utilizado por la interfaz. No se presenta como prueba manual exhaustiva de cada control.

Capturas y trazabilidad

Las imágenes se obtuvieron con la captura del emulador después de renderizar cada pantalla. Se verifican estados de registro, asignación, finalización, reseña y moderación durante la secuencia. Se conservaron RF-01 a RF-43 y RNF-01 a RNF-07 de la especificación original. EV identifica evidencias complementarias, sin redefinir los requisitos.

Reproducción

Comandos: ./gradlew :app:assembleDebug :app:testDebugUnitTest
Prueba de dispositivo: :app:connectedDebugAndroidTest con las clases DeliveryEvidenceTest, NavigationFlowTest y WelcomeFlowTest. Consultar docs/FASE_2.md en el repositorio.

Revisión humana pendiente del grupo

Antes de presentar: ejecutar el recorrido sugerido; explicar modelos, repositorio, ViewModel y roles; revisar las diferencias; confirmar el enlace/versionado en GitHub; verificar que las contribuciones de cada integrante estén registradas. No sustituir esa revisión por las pruebas automatizadas.

08 · REFERENCIAS

Fuentes y archivos de apoyo

Repositorio

github.com/JDFaFa/ServiNow

Guía de clase

Carlos Andrés Florez V. Arquitectura MVVM y Repositorios en Android. Universidad del Quindío.
Guía de ViewModels y repositorios

Documentación de herramientas

Hilt: configuración Gradle
Android Developers: Navigation Compose
Android Developers: pruebas de Compose

Archivos para estudiar

MainActivity.kt; features/community/CommunityScreen.kt; features/community/CommunityViewModel.kt; domain/model/Community.kt; domain/repository/CommunityRepository.kt; data/repository/InMemoryCommunityRepository.kt; di/RepositoryModule.kt; navigation/ServiNowNavigation.kt; features/welcome/WelcomeScreens.kt.

Pruebas y documentación

CommunityTest.kt; RepositoryBoundaryTest.kt; DeliveryEvidenceTest.kt; NavigationFlowTest.kt; WelcomeFlowTest.kt; docs/FASE_2.md; docs/ARQUITECTURA.md. Los nombres de carpetas reflejan la separación de responsabilidades; el código fuente permite revisar la implementación completa.